

Chattogram Diabetes Data Analysis

Data cleaning, regional distribution, age-group patterns, and clinical data preparation

Miskatul Anwar

2026-06-05

Table of contents

Chittagong Diabetes Data Analysis	6
Overview	6
Project Scope	6
Analysis Workflow	7
Data Cleaning and Standardization	7
Thana-Level Population Distribution and Age Group Analysis	7
Data Sources and Environment Variables	8
Current Dataset Preparation Logic	8
Regional Classification Used in the Analysis	9
Inside City Corporation	9
Outside City Corporation	9
Standardized Thana Mapping	9
Key Outputs	10
How to Render the Project	10
Project Files	11
Reproducibility Checklist	11
Next Steps	11
Data Cleaning and Standardization	12
Overview	12
Objectives	12
Expected Input and Output	12
Load Dependencies	13
Import Raw Dataset	13
Inspect Data Structure	13
Column Inventory	14
Remove Duplicate Columns	14
Remove Duplicate Records	14
Update the Original Dataset with Cleaned Data	15
Verify Merge Success	15
Define Export Path	16
Check Dataset Size After Initial Cleaning	16
Preview Updated Dataset	17
Convert Anthropometric Columns to Numeric Format	17
Standardize Fasting Column to Numerical Values	17

Verify Numerical Standardization	18
Remove Records with Missing Age	18
Export Cleaned Dataset	19
Cleaning Summary	19
Reproducibility Notes	19
Expected Output	20
Fasting, Body Composition, and Lifestyle Patterns by Age Group	21
Analysis Objective	21
Import necessary libraries	21
Set the visual theme	21
Load the dataset	22
Initial data overview	22
Shape of the dataset	22
Column names	22
Check required columns	23
Data types	24
Missing values summary	24
Duplicate rows	25
Descriptive statistics	25
Numerical summary	25
Categorical summary	25
Data cleaning and outlier handling	27
Quick sanity check after filtering	27
Feature engineering	28
Create age group labels	28
Create BMI category	28
Sample composition	29
Participant count by age group	29
Sex distribution across age groups	30
BMI category distribution by age group	30
Grouped descriptive analytics	31
Summary statistics by age group	31
Summary statistics by age group and sex	32
Correlation analysis	33
Overall correlation heatmap	33
Correlation heatmap for Age ≤ 30	34
Correlation heatmap for Age > 30	35
Distribution analysis	36
Fasting distribution overall	36
Fasting distribution by age group	37
BMI distribution overall	38
BMI distribution by age group	39

Density comparison of fasting by age group	40
Core relationship analysis	42
Age vs fasting	42
BMI vs fasting	44
Weight vs fasting	46
Height vs fasting	48
Categorical comparisons	49
Sex vs fasting	49
Sex vs BMI	50
Age group vs fasting	51
Age group vs BMI	53
Violin plot of fasting by age group	54
Diet-related analysis	55
Diet frequency table	55
Diet vs BMI	56
Diet vs fasting	56
Diet composition across age groups	57
Multivariable relationship analysis	58
Age and sex to fasting	58
BMI vs fasting faceted by age group	60
Weight vs fasting faceted by age group	60
Pairwise view of core variables	61
Group comparison tables	62
Mean comparison across age groups	62
Median comparison across age groups	63
Sex-wise mean fasting within each age group	63
BMI category by age group	64
Data Exploration and Fasting Glucose Quality Assessment	65
Data Loading and Preparation	65
Thana-Level Population Distribution and Age Group Analysis	74
Overview	74
Objectives	74
Dataset Loading and Environment Setup	74
Display Configuration	75
Initial Data Loading and Scope Filtering	75
City Corporation and Outside City Corporation Classification	76
Standardizing Thana Names	76
Reference List Size Check	77
Unique Thana Inspection	77
Age Group Creation	78
Total Population Count	80

Creating City Corporation and Outside City Corporation Subsets	80
Record Counts by Classification and Thana	81
Age Group Counts by Classification and Thana	81
City Corporation Summary	82
Outside City Corporation Summary	82
City Corporation Percentage Breakdown	84
Age Group Percentages Within City Corporation Thanas	85
Outside City Corporation Percentage Breakdown	85
Inside City Corporation Thana Percentage Breakdown	88
Global Population Share and Verification	89
Interpretation Guide	90
Notes for Reproducibility	91
Expected Output	91

Chittagong Diabetes Data Analysis

Data cleaning, regional distribution, age-group patterns, and clinical data preparation

Overview

This project documents a reproducible workflow for preparing and analyzing diabetes patient data from the Chittagong region. The current version of the project focuses on two major stages: cleaning and standardizing the raw clinical dataset, then analyzing the cleaned records by Thana, City Corporation classification, and age group.

The analysis pipeline is designed to support transparent reporting and future extension. Dataset paths are managed through environment variables, cleaned records are exported for downstream analysis, and location-based summaries are generated from the standardized dataset.

Project Scope

The project currently covers the following analytical areas:

- Cleaning raw diabetes patient records and removing duplicate entries.
- Updating fasting glucose values from a supplementary dataset using patient ID.
- Standardizing mixed-format clinical variables into analysis-ready numeric fields.
- Removing records with missing key analytical fields such as **Fasting**, **Age**, and **Thana** where required.
- Standardizing Thana names to reduce spelling and administrative-label inconsistencies.
- Separating records into City Corporation and outside City Corporation groups.
- Creating age groups using the threshold ≤ 30 and > 30 .
- Calculating population counts, Thana-wise distributions, and percentage breakdowns.
- Verifying whether the final regional grouping accounts for the full filtered dataset.

Analysis Workflow

The project is organized into a sequential workflow. Each stage produces outputs that support the next stage of analysis.

Data Cleaning and Standardization

The first stage prepares the raw diabetes dataset for analysis. It loads the primary dataset and supplementary fasting-value dataset from environment-defined paths, removes duplicate columns and duplicate patient records, updates fasting values, standardizes numeric fields, and exports the cleaned dataset.

Key tasks in this stage include:

- Loading the raw Excel dataset from `DATASET_PATH`.
- Removing duplicate columns.
- Deduplicating records by ID, keeping the last available record.
- Loading the supplementary file from `NEW_PATH`.
- Updating the `Fasting` column by matching records on ID.
- Converting BMI, `Weight` (KG), and `Height` (CM) to numeric values.
- Extracting numeric values from mixed-format `Fasting` entries.
- Removing records with missing fasting glucose or age values.
- Saving the cleaned dataset to `CLEANED_DATASET_PATH`.

The complete documentation for this stage is available in:

```
data_cleaning_and_standardization.qmd
```

Thana-Level Population Distribution and Age Group Analysis

The second stage uses the cleaned dataset to examine the regional distribution of patient records. It focuses on Thana-level classification, City Corporation grouping, age-group segmentation, and percentage-based summaries.

Key tasks in this stage include:

- Loading the cleaned dataset from `CLEANED_DATASET_PATH`.
- Removing Thanas outside the selected analysis scope.
- Dropping records with missing `Thana` values.
- Mapping alternative Thana names to standardized labels.
- Defining Thanas inside and outside the City Corporation.
- Creating the `Age Group` variable using `<=30` and `>30` categories.

- Calculating total population records after filtering.
- Summarizing records inside the City Corporation.
- Summarizing records outside the City Corporation.
- Calculating Thana-level and age-group percentages.
- Running a final verification check to identify unmapped or unclassified records.

The complete documentation for this stage is available in:

```
thana_population_distribution_analysis.qmd
```

Data Sources and Environment Variables

This project avoids hard-coded file paths. The workflow expects file locations to be provided through environment variables.

Required variables:

```
DATASET_PATH
NEW_PATH
CLEANED_DATASET_PATH
```

These variables should be defined in a `.env` file or made available directly in the execution environment before rendering the project files.

Current Dataset Preparation Logic

The latest data preparation workflow uses the following assumptions and rules:

- ID is treated as the unique patient identifier for deduplication.
- If multiple records share the same ID, the last occurrence is retained.
- The supplementary dataset is used to update fasting glucose values in the primary dataset.
- Fasting glucose values may contain units or mixed text, so the first numeric value is extracted.
- Records without valid fasting glucose values are removed from the cleaned dataset.
- Records without age values are removed from the cleaned dataset.
- Thana names are standardized before regional analysis.
- Records without valid Thana values are removed before Thana-level analysis.

Regional Classification Used in the Analysis

The current regional analysis separates Thanas into two groups: inside the City Corporation and outside the City Corporation.

Inside City Corporation

The current City Corporation list includes:

Bayejid Bostami, Chandgaon, Bakalia, Kotwali, Double Mooring, Patenga, Chittagong Port, Halishahar, Pahartali, Khulshi, Panchlaish

Outside City Corporation

The current outside City Corporation list includes:

Mirsharai, Fatikchhari, Raozan, Sitakunda, Hathazari, Boalkhali, Patiya, Chandanaish, Sandwip, Rangunia, Anowara, Satkania, Banshkhali, Lohagara

Standardized Thana Mapping

The regional analysis standardizes alternative Thana names before grouping. Examples include:

Firingibazar -> Kotwali
EPZ -> Chittagong Port
Lalkhan Bazar -> Khulshi
Patharghata -> Kotwali
Kaptai -> Rangunia
Fouzdarhat -> Sitakunda
New Mooring -> Chittagong Port
Agrabad -> Double Mooring
Karnaphuli -> Patiya
Bhujpur -> Fatikchhari
Chawk Bazar -> Panchlaish
Bandar -> Chittagong Port
Anwara -> Anowara
Sadarghat -> Patiya

```
Akbar Shah -> Pahartali  
Bayazid Bostami -> Bayejid Bostami  
Bandarban Sadar -> Satkania
```

Key Outputs

The current project generates the following outputs:

- A cleaned Excel dataset saved to the path defined by `CLEANED_DATASET_PATH`.
- A verified column inventory after duplicate column removal.
- Deduplicated patient records based on `ID`.
- Updated fasting glucose values from the supplementary dataset.
- Numeric versions of fasting glucose, BMI, weight, and height fields.
- Total population count after cleaning and filtering.
- City Corporation and outside City Corporation record counts.
- Thana-wise population counts.
- Age group counts by Thana.
- Percentage breakdowns within City Corporation and outside City Corporation groups.
- Global population share by City Corporation and outside-City-Corporation Thanas.
- A final verification check for unmapped or unclassified Thana records.

How to Render the Project

From the project root, render the website with:

```
quarto render
```

To preview the website locally, use:

```
quarto preview
```

Before rendering, confirm that the `.env` file is available and that all required dataset paths are valid.

Project Files

The latest documentation files are:

```
index.qmd
data_cleaning_and_standardization.qmd
thana_population_distribution_analysis.qmd
```

Suggested website order:

```
index.qmd
Data Cleaning and Standardization
Thana-Level Population Distribution and Age Group Analysis
```

Reproducibility Checklist

Before running the full project, verify that:

- The `.env` file exists or environment variables are already configured.
- `DATASET_PATH` points to the raw diabetes dataset.
- `NEW_PATH` points to the supplementary fasting-value dataset.
- `CLEANED_DATASET_PATH` points to the intended cleaned output file.
- The raw and supplementary datasets contain the required columns.
- Python dependencies are installed, including `pandas`, `numpy`, `python-dotenv`, and `openpyxl`.
- The data cleaning stage is rendered before the regional analysis stage.

Next Steps

The current project website can be extended with additional analysis pages, such as:

- Exploratory analysis of clinical variables.
- Diabetes risk factor profiling by age group and region.
- BMI, fasting glucose, and anthropometric distribution analysis.
- City Corporation versus outside City Corporation comparison.
- Visual dashboards or maps for Thana-level patterns.
- Model-ready feature engineering and predictive modeling documentation.

Data Cleaning and Standardization

Overview

This document describes a reproducible data cleaning and standardization workflow for a diabetes patient dataset. The purpose of this workflow is to transform raw patient records into a cleaner and more analysis-ready dataset by removing duplicate fields and records, updating fasting glucose values from a supplementary file, standardizing mixed-format numeric columns, and exporting the cleaned dataset for downstream analysis.

The workflow relies on environment variables to locate the raw dataset, supplementary update file, and final cleaned output path. This approach avoids hard-coding local file locations and makes the pipeline easier to reuse across different environments.

Objectives

The main objectives of this cleaning process are to:

- Load the raw diabetes dataset from a configured environment path.
- Inspect the structure and column inventory of the dataset.
- Remove duplicate columns created during data collection or export.
- Remove duplicate patient records using ID as the unique identifier.
- Update the primary dataset with corrected or newly available fasting glucose values from a supplementary dataset.
- Convert selected anthropometric variables to numeric format.
- Extract numeric fasting glucose values from mixed text formats.
- Remove records that are missing key analytical fields such as **Fasting** and **Age**.
- Export the cleaned dataset to the configured output path.

Expected Input and Output

This workflow expects the following environment variables to be available:

- `DATASET_PATH`: path to the primary raw Excel dataset.

- `NEW_PATH`: path to the supplementary Excel dataset containing updated fasting glucose values.
- `CLEANED_DATASET_PATH`: path where the cleaned Excel dataset will be saved.

The expected output is a cleaned Excel dataset containing standardized patient records suitable for subsequent analysis, modeling, or reporting.

Load Dependencies

The workflow begins by importing the required Python libraries. The `os` module is used to read environment variables, `pandas` is used for data loading and transformation, and `load_dotenv` is used to load variables from a `.env` file.

```
import os
import pandas as pd
from dotenv import load_dotenv
```

Import Raw Dataset

The environment variables are loaded first. The raw dataset path is then retrieved from `DATASET_PATH`, and the Excel file is imported into a pandas DataFrame.

```
load_dotenv()
dataset_path = os.getenv("DATASET_PATH")
df = pd.read_excel(dataset_path)
```

Inspect Data Structure

A preview of the first few rows is displayed to understand the structure of the dataset, verify that the file was loaded correctly, and inspect the initial format of the variables.

```
df.head()
```

	ID	Entry Date	Age	Sex	Address	Height (CM)	Weight (KG)	Fasting Glucose (mg/dL)
0	K-225859	2025-02-25 00:00:00	50.0	F	MIRESHARAI CTG	147CM	44 KG	17.0M
1	K-225401	2025-02-11 00:00:00	35.0	M	LOHAGORA CTG	160CM	62 KG	10.0 M
2	K-225407	2025-02-11 00:00:00	55.0	M	SANDWIP CTG	164CM	61KG	15.5M

	ID	Entry Date	Age	Sex	Address	Height (CM)	Weight (KG)	Fasting
3	K-225413	2025-02-11 00:00:00	50.0	M	ANOWARA CTG	151CM	52 KG	22.7M
4	K-225655	2025-02-19 00:00:00	57.0	M	MIRESHARAI CTG	168CM	76KG	7.6MM

Column Inventory

The column names are collected and printed. This step provides a quick inventory of all variables available in the raw dataset and helps identify potential duplicate columns, inconsistent naming, or unexpected fields.

```
cols = df.columns.tolist()
for col in cols:
    print(col, end=", ")
print(f"\n\nTotal {len(cols)} columns")
```

ID, Entry Date, Age, Sex, Address, Height (CM), Weight (KG), Fasting, ABF, Uring Color Fasting
District, Thana,

Total 23 columns

Remove Duplicate Columns

Duplicate columns are removed from the dataset. This can happen when Excel files are exported, merged, or manually edited. Keeping only the first occurrence of each column name prevents ambiguity during later transformations.

```
df = df.loc[:, ~df.columns.duplicated()]
```

Remove Duplicate Records

Duplicate patient records are removed using ID as the unique patient identifier. When multiple records share the same ID, the last occurrence is retained. This assumes that later entries are more complete or more recently updated.

```
df.drop_duplicates(subset="ID", keep="last", inplace=True)
df.__len__()
```

44136

Update the Original Dataset with Cleaned Data

A supplementary dataset is loaded from the path stored in `NEW_PATH`. This file is used to update the `Fasting` column in the primary dataset.

Before merging, duplicate records are also removed from the supplementary dataset using the same ID-based rule. Copies of the `ID` and `Fasting` columns are created for comparison and verification purposes before the update is applied.

```
new_file = os.getenv("NEW_PATH")
new_df = pd.read_excel(new_file)
new_df.drop_duplicates(subset="ID", keep="last", inplace=True)

# Create index copies for comparison before merge
df_before_fasting = df[["ID", "Fasting"]].copy()
new_df_fasting = new_df[["ID", "Fasting"]].copy()
```

The update is performed by temporarily setting `ID` as the index in both datasets. The `Fasting` values in the primary dataset are then updated using matching `ID` values from the supplementary dataset. After the update, the index is reset so that `ID` returns to a regular column.

```
# Perform merge using ID-based index
df.set_index("ID", inplace=True)
new_df.set_index("ID", inplace=True)
df.update(new_df["Fasting"])
df.reset_index(inplace=True)
```

Verify Merge Success

The following checks confirm whether the `Fasting` column contains values after the merge and how many records still have missing fasting values. This provides a simple quality-control checkpoint before further standardization.

```
# Count records with Fasting values before and after merge
print("=== Fasting Column Update Verification ===\n")
print(f"Records with Fasting values (after merge): {df['Fasting'].notna().sum()}")
print(f"Records with missing Fasting: {df['Fasting'].isna().sum()}")
print(f"\nTotal records: {len(df)}")
```

=== Fasting Column Update Verification ===

Records with Fasting values (after merge): 43783

Records with missing Fasting: 353

Total records: 44136

A small sample of the updated ID and Fasting values is displayed to visually inspect whether the merged column appears correctly aligned with patient records.

```
# Show sample of merged data to verify alignment
merge_sample = df[["ID", "Fasting"]].head(10)
print("\nSample of merged Fasting column:")
print(merge_sample)
```

Sample of merged Fasting column:

	ID	Fasting
0	K-225859	17.0MMOL
1	K-225401	10.0 MMOL
2	K-225407	15.5MMOL
3	K-225413	22.7MMOL
4	K-225655	7.6MMOL
5	K-225638	5.5MMOL
6	K-225634	24.8MMOL
7	K-225642	14.0MMOL
8	K-225632	11.5MMOL
9	K-225636	9.3MMOL

Define Export Path

The cleaned dataset output path is retrieved from the `CLEANED_DATASET_PATH` environment variable. This path is used at the end of the workflow to save the final cleaned Excel file.

```
clean_dataset_path = os.getenv("CLEANED_DATASET_PATH")
```

Check Dataset Size After Initial Cleaning

The number of records remaining after duplicate removal and fasting-value update is checked before additional transformations are applied.


```
df.__len__()
```

44136

Preview Updated Dataset

The dataset is previewed again to inspect the current state after duplicate removal and fasting-value update.

```
df.head()
```

	ID	Entry Date	Age	Sex	Address	Height (CM)	Weight (KG)	Fasting
0	K-225859	2025-02-25 00:00:00	50.0	F	MIRESHARAI CTG	147CM	44 KG	17.0M
1	K-225401	2025-02-11 00:00:00	35.0	M	LOHAGORA CTG	160CM	62 KG	10.0 M
2	K-225407	2025-02-11 00:00:00	55.0	M	SANDWIP CTG	164CM	61KG	15.5M
3	K-225413	2025-02-11 00:00:00	50.0	M	ANOWARA CTG	151CM	52 KG	22.7M
4	K-225655	2025-02-19 00:00:00	57.0	M	MIRESHARAI CTG	168CM	76KG	7.6MM

Convert Anthropometric Columns to Numeric Format

The columns BMI, Weight (KG), and Height (CM) are converted to numeric data types. Any non-convertible values are coerced to missing values using `errors="coerce"`.

This step ensures that these fields can be used reliably in later statistical analysis, visualization, or machine learning workflows.

```
cols = ["BMI", "Weight (KG)", "Height (CM)"]

for col in cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")
```

Standardize Fasting Column to Numerical Values

The original Fasting column may contain mixed-format values, such as numbers combined with units or text. A regular expression is used to extract the first numeric value from each entry.

For example, a value such as 10.0MMOL is converted to 10.0. Records without a valid numeric fasting value are treated as missing.

```
import re
def extract_numerical_value(value):
    if pd.isna(value):
        return None
    match = re.search(r"(\d+\.?(\d*))", str(value))
    return float(match.group(1)) if match else None
df["Fasting"] = df["Fasting"].apply(extract_numerical_value)
df["Fasting"] = df["Fasting"].astype(float)
df.dropna(subset=["Fasting"], inplace=True)
```

Verify Numerical Standardization

This verification step confirms how many records have valid numerical fasting values after extraction and type conversion. It also reports the number of missing fasting values and the remaining dataset size.

```
print("=== Fasting Column Standardization Verification ===\n")
print(f"Records with numerical Fasting values: {df['Fasting'].notna().sum()}")
print(f"Records with missing Fasting: {df['Fasting'].isna().sum()}")
print(f"\nTotal records: {len(df)}")
```

```
=== Fasting Column Standardization Verification ===
```

```
Records with numerical Fasting values: 43777
```

```
Records with missing Fasting: 0
```

```
Total records: 43777
```

Remove Records with Missing Age

Records with missing Age values are removed. Age is typically a key variable in diabetes-related analysis, and removing missing values here ensures that downstream analysis can rely on complete age information.

```
df.dropna(subset=["Age"], inplace=True)
```

Export Cleaned Dataset

The final dataset is previewed and then exported as an Excel file to the path defined by `CLEANED_DATASET_PATH`.

```
df.head()  
df.to_excel(clean_dataset_path, index=False)
```

Cleaning Summary

This cleaning workflow performs the following transformations:

- Loads the primary diabetes patient dataset from an environment-defined path.
- Removes duplicate columns from the raw dataset.
- Removes duplicate patient records using ID, keeping the last occurrence.
- Loads a supplementary dataset and updates the primary **Fasting** values by matching on ID.
- Verifies the fasting-value update through count summaries and sample inspection.
- Converts **BMI**, **Weight (KG)**, and **Height (CM)** to numeric format.
- Extracts numeric values from mixed-format fasting glucose entries.
- Removes records missing valid fasting glucose values.
- Removes records missing age values.
- Saves the cleaned dataset for downstream analysis.

Reproducibility Notes

Before rendering this document, ensure that:

- A `.env` file exists or the required environment variables are already available in the execution environment.
- `DATASET_PATH` points to the raw Excel dataset.
- `NEW_PATH` points to the supplementary Excel dataset used for fasting-value updates.
- `CLEANED_DATASET_PATH` points to the desired cleaned dataset output location.
- The input datasets contain the required columns: **ID**, **Fasting**, **Age**, **BMI**, **Weight (KG)**, and **Height (CM)**.
- Required Python packages are installed: `pandas`, `python-dotenv`, and an Excel engine such as `openpyxl`.

Expected Output

After running this document successfully, the output should be a cleaned Excel dataset with:

- Duplicate columns removed.
- Duplicate patient IDs resolved.
- Updated fasting glucose values merged from the supplementary dataset.
- Numeric values extracted from the **Fasting** column.
- Anthropometric variables converted to numeric format.
- Records with missing **Fasting** or **Age** removed.
- Final cleaned data saved to **CLEANED_DATASET_PATH**.

Fasting, Body Composition, and Lifestyle Patterns by Age Group

Analysis Objective

This report explores how fasting measurements, BMI, body size, sex, and lifestyle-related variables vary across two age groups:

- Age $\leq$ 30
- Age $>$ 30

The objective is to identify descriptive patterns, compare distributions, and examine how anthropometric and metabolic variables behave across these two age-based segments.

Import necessary libraries

```
import os
import numpy as np
import pandas as pd
import seaborn as sns
from dotenv import load_dotenv
```

Set the visual theme

```
sns.set_theme(
    style="whitegrid",
    context="notebook",
    rc={"figure.figsize": (8, 5)}
)
```

Load the dataset

```
load_dotenv()

dataset_path = os.getenv("CLEANED_DATASET_PATH")
if dataset_path is None:
    raise ValueError("CLEANED_DATASET_PATH is not set in the environment.")

df = pd.read_excel(dataset_path)
df.head()
```

	ID	Entry Date	Age	Sex	Address	Height (CM)	Weight (KG)	Fastin
0	K-225859	2025-02-25 00:00:00	50.0	F	MIRESHARAI CTG	NaN	NaN	17.0
1	K-225401	2025-02-11 00:00:00	35.0	M	LOHAGORA CTG	NaN	NaN	10.0
2	K-225407	2025-02-11 00:00:00	55.0	M	SANDWIP CTG	NaN	NaN	15.5
3	K-225413	2025-02-11 00:00:00	50.0	M	ANOWARA CTG	NaN	NaN	22.7
4	K-225655	2025-02-19 00:00:00	57.0	M	MIRESHARAI CTG	NaN	NaN	7.6

Initial data overview

Shape of the dataset

```
df.shape
```

```
(43777, 23)
```

Column names

```
df.columns.tolist()
```

```
['ID',
 'Entry Date',
 'Age',
 'Sex',
```

```
'Address',
'Height (CM)',
'Weight (KG)',
'Fasting',
'ABF',
'Uring Color Fasting',
'Uring Color ABF',
'Acitone',
'Albumin',
'Diet',
'Height (CM).1',
'Weight (KG).1',
'BMI',
'Normalized Address',
'Division',
'District',
'City Corporation',
'Sub-District',
'Thana']
```

Check required columns

```
required_columns = [
    "Age",
    "Sex",
    "Height (CM)",
    "Weight (KG)",
    "BMI",
    "Fasting"
]

missing_columns = [col for col in required_columns if col not in df.columns]

if missing_columns:
    raise ValueError(f"Missing required columns: {missing_columns}")
else:
    print("All required columns are present.")
```

All required columns are present.

Data types

```
df.dtypes
```

```
ID                object
Entry Date        object
Age              float64
Sex               str
Address           str
Height (CM)       float64
Weight (KG)       float64
Fasting          float64
ABF              object
Uring Color Fasting  object
Uring Color ABF   object
Acitone           object
Albumin           object
Diet              object
Height (CM).1     float64
Weight (KG).1     float64
BMI              float64
Normalized Address str
Division          str
District          str
City Corporation  str
Sub-District      str
Thana             str
dtype: object
```

Missing values summary

```
df[required_columns].isnull().sum().sort_values(ascending=False)
```

```
Height (CM)    32743
Weight (KG)    32652
BMI            9
Age            0
Sex            0
Fasting        0
dtype: int64
```


Duplicate rows

```
df.duplicated().sum()
```

```
np.int64(0)
```

Descriptive statistics

Numerical summary

```
df[["Age", "Height (CM)", "Weight (KG)", "BMI", "Fasting"]].describe()
```

	Age	Height (CM)	Weight (KG)	BMI	Fasting
count	43777.000000	11034.000000	11125.000000	43768.000000	43777.000000
mean	47.598006	158.110413	64.855092	41.582140	11.537003
std	12.155562	27.258622	60.133519	2868.743566	22.558498
min	3.000000	5.300000	0.067000	0.000000	0.000000
25%	39.000000	151.000000	56.000000	23.111111	7.500000
50%	48.000000	158.000000	64.000000	25.566107	9.900000
75%	55.000000	165.000000	72.000000	28.344671	14.000000
max	110.000000	1688.000000	6210.700000	600000.000000	3407.000000

Categorical summary

```
categorical_columns = ["Sex"] + (["Diet"] if "Diet" in df.columns else [])

for col in categorical_columns:
    print(f"\nValue counts for: {col}")
    print(df[col].value_counts(dropna=False))
```

```
Value counts for: Sex
Sex
F    23230
```

```

M      20545
0        2
Name: count, dtype: int64

```

Value counts for: Diet

```

Diet
NaN      32008
6         2788
3         2659
4         2516
5         2131
2         1172
7          418
1          34
REGNANC   5
8          5
06 (P.D)   3
N/S        3
07 (P.D)   3
02NEW PA   2
07(P.D)    2
9          2
06  PPREG   1
04NEW PA   1
0595/60    1
6-         1
03(P.D)    1
....       1
.....     1
2000       1
1400       1
2200       1
K          1
47         1
35         1
04-        1
6.1        1
06160/100  1
53         1
120/80     1
6(P.D)     1
-----   1
NEXT TIME  1

```

```

58          1
06 (PD)      1
04\         1
39          1
16          1
Name: count, dtype: int64

```

Data cleaning and outlier handling

The goal here is to remove implausible or extreme records that may distort the analysis.

```

df = df.drop_duplicates().copy()

df = df[
    ~(
        (df["BMI"] >= 60) |
        (df["BMI"] <= 15) |
        (df["Weight (KG)"] >= 500) |
        (df["Weight (KG)"] <= 30) |
        (df["Height (CM)"] > 250) |
        (df["Height (CM)"] <= 100) |
        (df["Fasting"] > 50)
    )
].copy()

```

Quick sanity check after filtering

```
df[["Age", "Height (CM)", "Weight (KG)", "BMI", "Fasting"]].describe()
```

	Age	Height (CM)	Weight (KG)	BMI	Fasting
count	43517.000000	10945.000000	11027.000000	43508.000000	43517.000000
mean	47.618561	158.030882	64.392128	25.945947	11.162435
std	12.127001	8.886475	11.665086	4.197449	4.606895
min	3.000000	121.000000	31.000000	15.010751	0.000000
25%	39.000000	151.000000	56.000000	23.138507	7.500000
50%	48.000000	158.000000	64.000000	25.593737	9.800000
75%	55.000000	165.000000	72.000000	28.326580	13.900000
max	110.000000	189.000000	135.000000	57.471264	38.000000

Feature engineering

Create age group labels

```
df["Age Group"] = np.where(df["Age"] <= 30, "<=30", ">30")
df["Age Group"].value_counts()
```

```
Age Group
>30      39893
<=30     3624
Name: count, dtype: int64
```

Create BMI category

```
bmi_bins = [0, 18.5, 25, 30, np.inf]
bmi_labels = ["Underweight", "Normal", "Overweight", "Obese"]

df["BMI Category"] = pd.cut(
    df["BMI"],
    bins=bmi_bins,
    labels=bmi_labels,
    right=False
)

df["BMI Category"].value_counts(dropna=False)
```

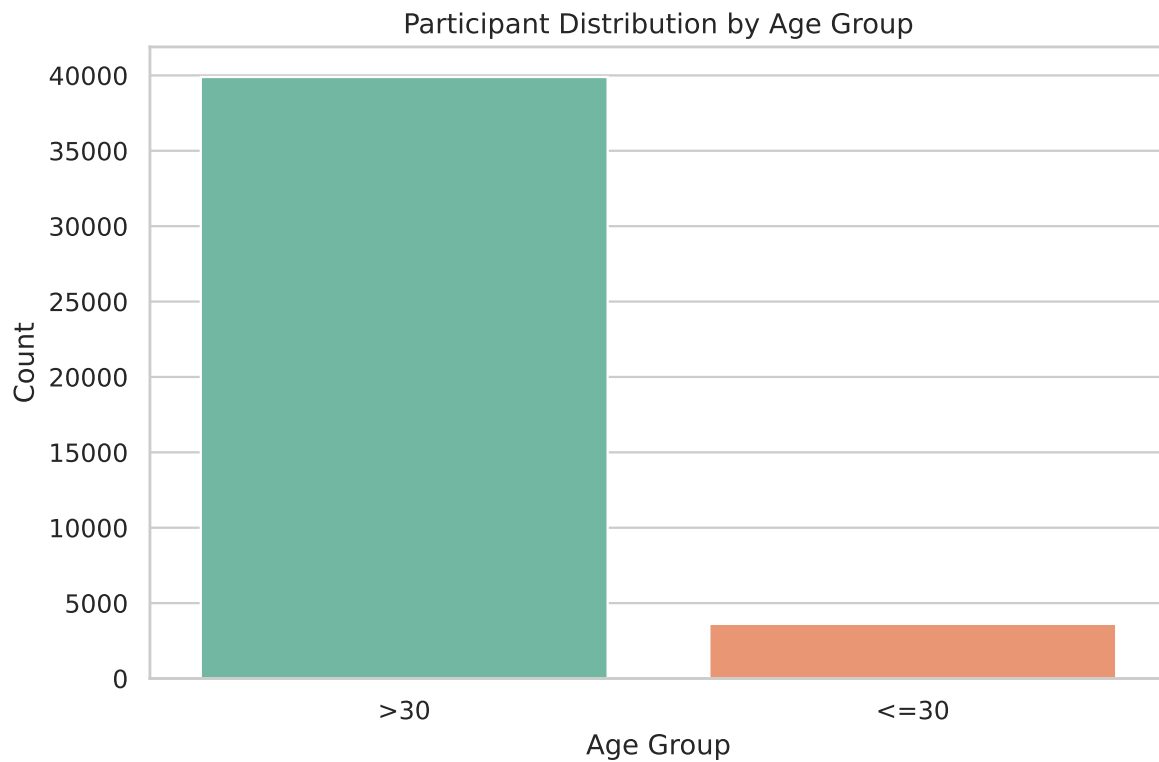
```
BMI Category
Normal      18123
Overweight  17763
Obese       6639
Underweight  983
NaN          9
Name: count, dtype: int64
```

Sample composition

Participant count by age group

```
ax = sns.countplot(data=df, x="Age Group", hue="Age Group", palette="Set2", legend=False)
ax.set(
    xlabel="Age Group",
    ylabel="Count",
    title="Participant Distribution by Age Group"
)
```

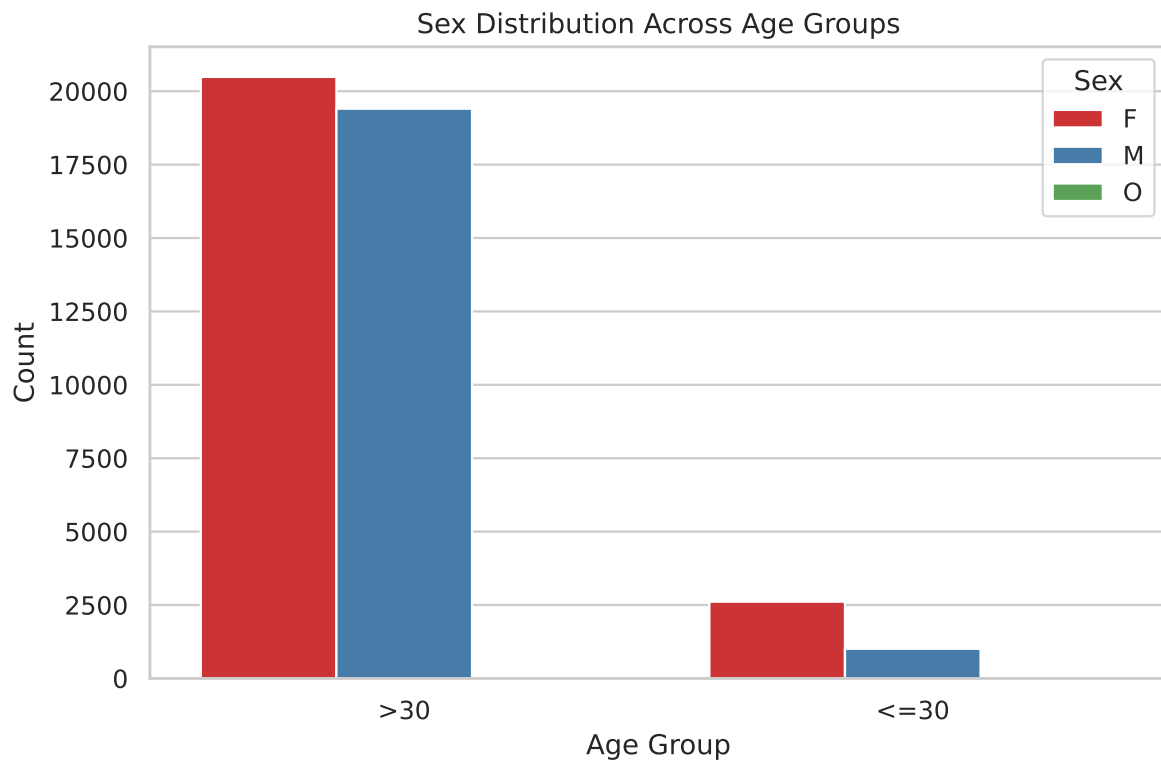
```
[Text(0.5, 0, 'Age Group'),
 Text(0, 0.5, 'Count'),
 Text(0.5, 1.0, 'Participant Distribution by Age Group')]
```



Sex distribution across age groups

```
ax = sns.countplot(data=df, x="Age Group", hue="Sex", palette="Set1")
ax.set(
    xlabel="Age Group",
    ylabel="Count",
    title="Sex Distribution Across Age Groups"
)
```

```
[Text(0.5, 0, 'Age Group'),
 Text(0, 0.5, 'Count'),
 Text(0.5, 1.0, 'Sex Distribution Across Age Groups')]
```



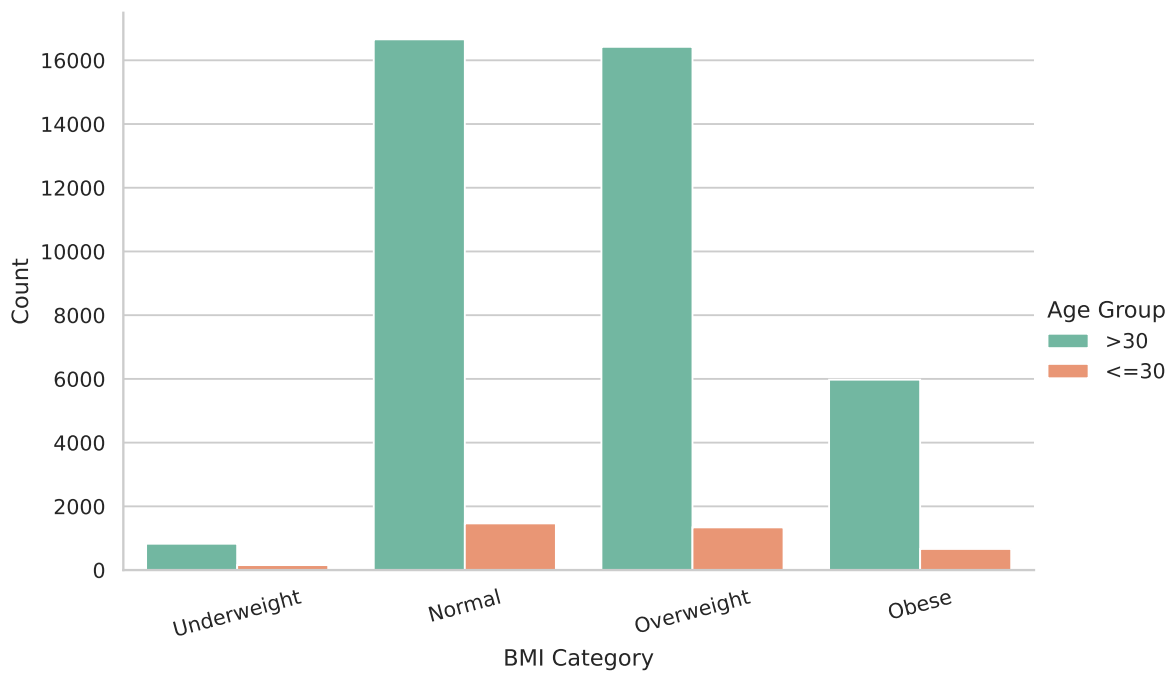
BMI category distribution by age group

```

g = sns.catplot(
    data=df,
    x="BMI Category",
    hue="Age Group",
    kind="count",
    palette="Set2",
    height=5,
    aspect=1.6
)

g.set_axis_labels("BMI Category", "Count")
g.set_titles("BMI Category Distribution by Age Group")
g.ax.tick_params(axis="x", rotation=15)

```



Grouped descriptive analytics

Summary statistics by age group

```
group_summary = (
    df.groupby("Age Group")[["Age", "Height (CM)", "Weight (KG)", "BMI", "Fasting"]]
    .agg(["count", "mean", "median", "std", "min", "max"])
    .round(2)
)

group_summary
```

Age Group	Age						Height (CM)				...		BMI	std
	count	mean	median	std	min	max	count	mean	median	std	...	...	median	
<=30	3624	26.72	28.0	3.57	3.0	30.0	896	156.88	155.0	8.53	...	...	25.61	4.1
>30	39893	49.52	49.0	10.77	31.0	110.0	10049	158.13	158.0	8.91	...	...	25.59	4.1

Summary statistics by age group and sex

```
group_sex_summary = (
    df.groupby(["Age Group", "Sex"])[["Height (CM)", "Weight (KG)", "BMI", "Fasting"]]
    .agg(["count", "mean", "median", "std"])
    .round(2)
)

group_sex_summary
```

Age Group	Sex	Height (CM)				Weight (KG)				BMI		
		count	mean	median	std	count	mean	median	std	count	mean	median
<=30	F	623	153.20	153.0	5.62	631	62.66	61.0	13.08	2614	26.66	26.25
	M	273	165.27	166.0	8.09	278	66.84	66.0	13.75	1010	24.43	24.17
	F	5002	151.73	152.0	5.76	5055	61.54	61.0	11.13	20485	26.82	26.48
>30	M	5047	164.48	165.0	6.65	5063	67.32	67.0	11.12	19397	25.00	24.80
	O	0	NaN	NaN	NaN	0	NaN	NaN	NaN	2	25.58	25.58

Correlation analysis

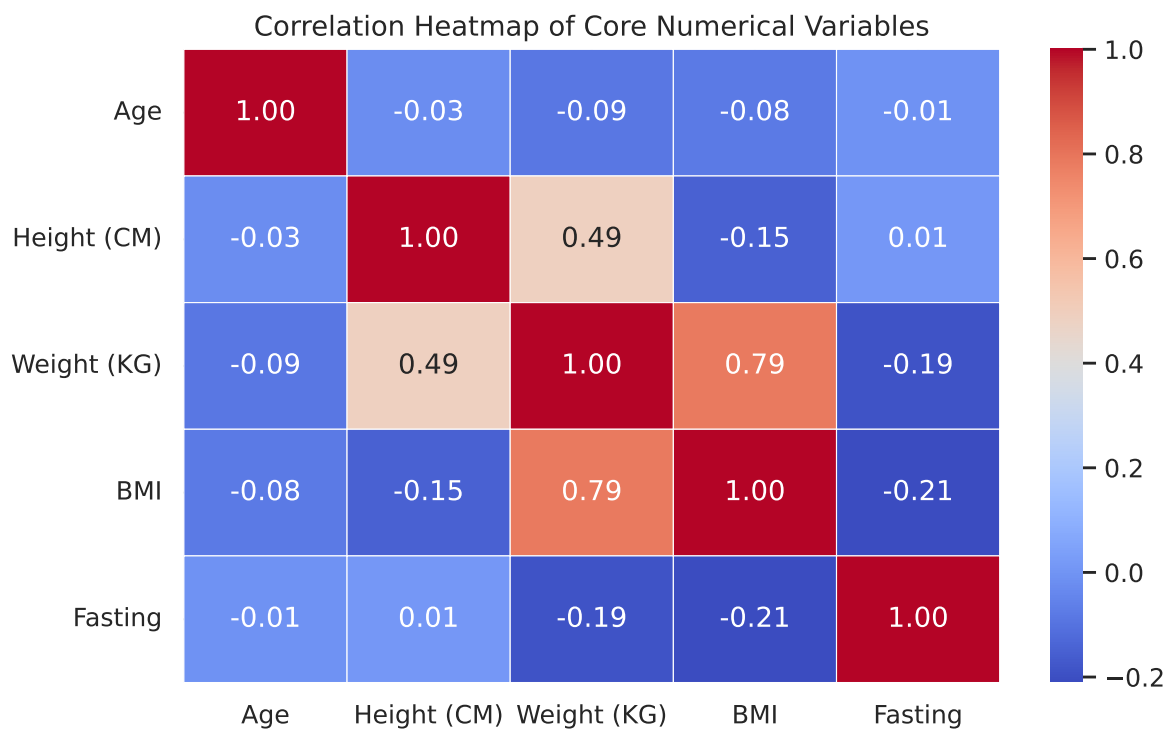
Overall correlation heatmap

```
corr = df[["Age", "Height (CM)", "Weight (KG)", "BMI", "Fasting"]].corr(numeric_only=True)

ax = sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)

ax.set_title("Correlation Heatmap of Core Numerical Variables")
```

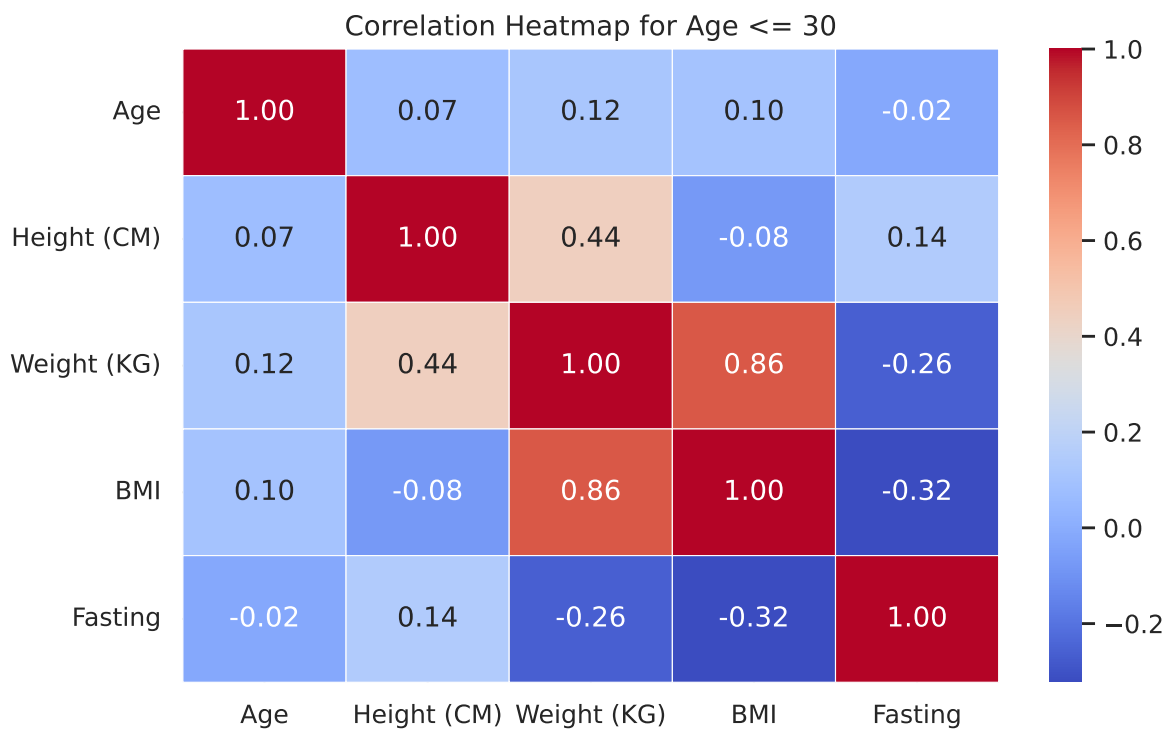
```
Text(0.5, 1.0, 'Correlation Heatmap of Core Numerical Variables')
```



Correlation heatmap for Age <= 30

```
corr_young = (  
    df.loc[df["Age Group"] == "<=30", ["Age", "Height (CM)", "Weight (KG)", "BMI", "Fasting"]  
    .corr(numeric_only=True)  
)  
  
ax = sns.heatmap(  
    corr_young,  
    annot=True,  
    cmap="coolwarm",  
    fmt=".2f",  
    linewidths=0.5  
)  
  
ax.set_title("Correlation Heatmap for Age <= 30")
```

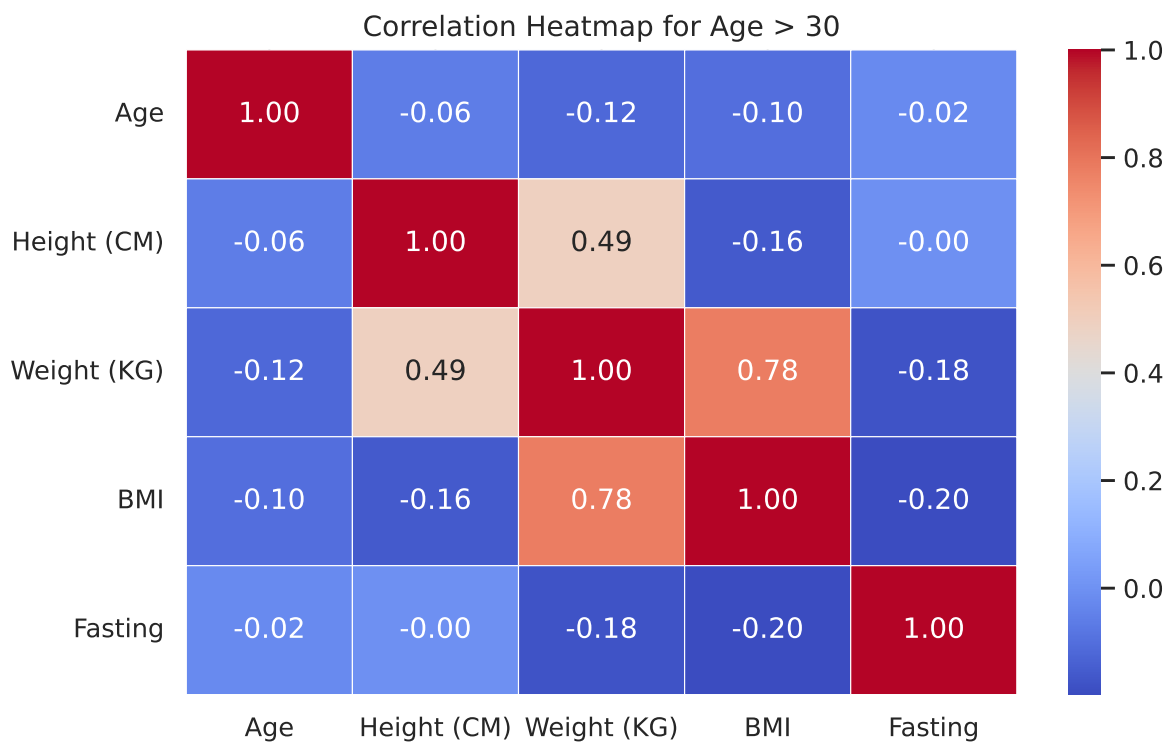
```
Text(0.5, 1.0, 'Correlation Heatmap for Age <= 30')
```



Correlation heatmap for Age > 30

```
corr_older = (  
    df.loc[df["Age Group"] == ">30", ["Age", "Height (CM)", "Weight (KG)", "BMI", "Fasting"]  
    .corr(numeric_only=True)  
)  
  
ax = sns.heatmap(  
    corr_older,  
    annot=True,  
    cmap="coolwarm",  
    fmt=".2f",  
    linewidths=0.5  
)  
  
ax.set_title("Correlation Heatmap for Age > 30")
```

```
Text(0.5, 1.0, 'Correlation Heatmap for Age > 30')
```

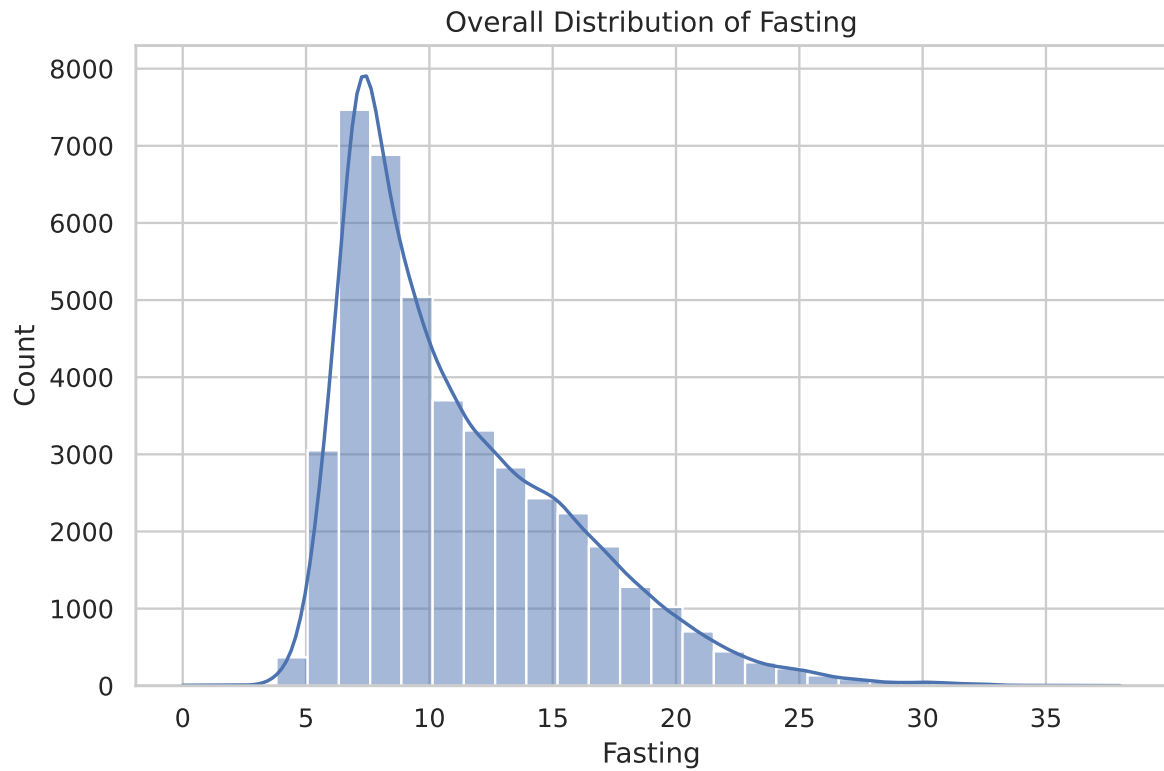


Distribution analysis

Fasting distribution overall

```
ax = sns.histplot(  
    data=df,  
    x="Fasting",  
    kde=True,  
    bins=30  
)  
  
ax.set(  
    xlabel="Fasting",  
    ylabel="Count",  
    title="Overall Distribution of Fasting"  
)
```

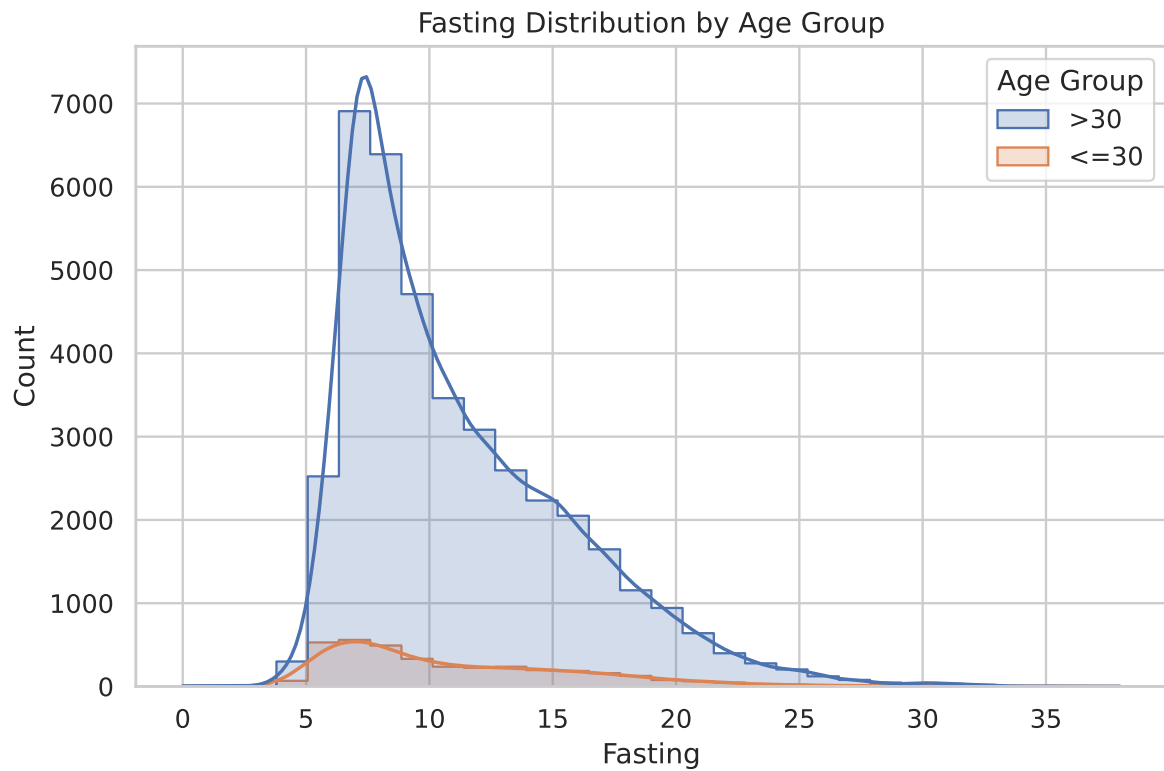
```
[Text(0.5, 0, 'Fasting'),  
Text(0, 0.5, 'Count'),  
Text(0.5, 1.0, 'Overall Distribution of Fasting')]
```



Fasting distribution by age group

```
ax = sns.histplot(  
    data=df,  
    x="Fasting",  
    hue="Age Group",  
    kde=True,  
    bins=30,  
    element="step",  
    stat="count",  
    common_norm=False  
)  
  
ax.set(  
    xlabel="Fasting",  
    ylabel="Count",  
    title="Fasting Distribution by Age Group"  
)
```

```
[Text(0.5, 0, 'Fasting'),
Text(0, 0.5, 'Count'),
Text(0.5, 1.0, 'Fasting Distribution by Age Group')]
```

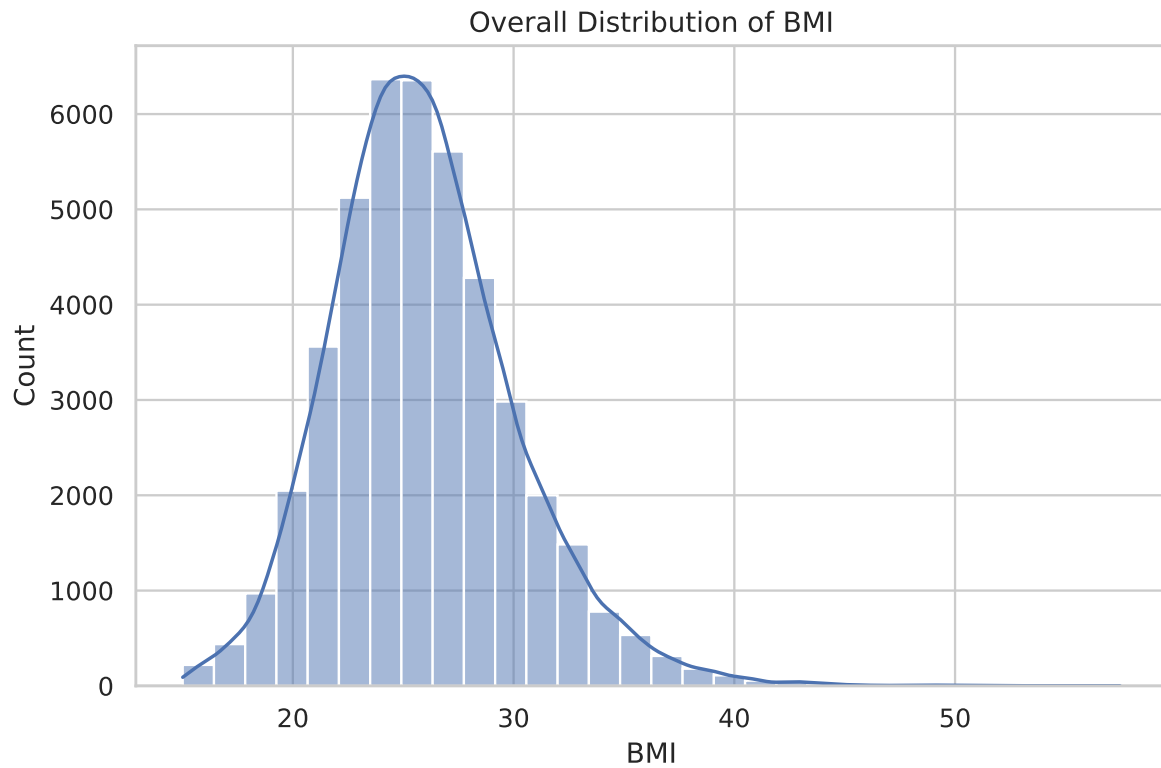


BMI distribution overall

```
ax = sns.histplot(
    data=df,
    x="BMI",
    kde=True,
    bins=30
)

ax.set(
    xlabel="BMI",
    ylabel="Count",
    title="Overall Distribution of BMI"
)
```

```
[Text(0.5, 0, 'BMI'),  
Text(0, 0.5, 'Count'),  
Text(0.5, 1.0, 'Overall Distribution of BMI')]
```

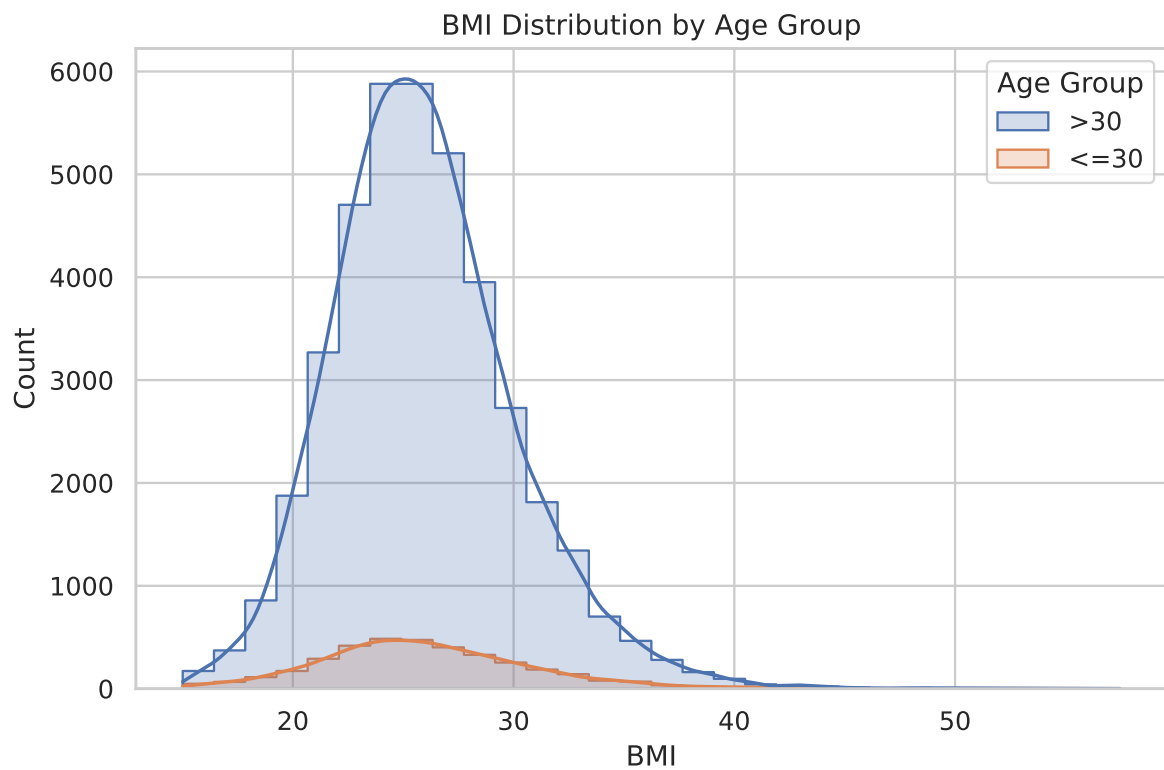


BMI distribution by age group

```
ax = sns.histplot(  
    data=df,  
    x="BMI",  
    hue="Age Group",  
    kde=True,  
    bins=30,  
    element="step",  
    stat="count",  
    common_norm=False  
)
```

```
ax.set(
    xlabel="BMI",
    ylabel="Count",
    title="BMI Distribution by Age Group"
)
```

```
[Text(0.5, 0, 'BMI'),
 Text(0, 0.5, 'Count'),
 Text(0.5, 1.0, 'BMI Distribution by Age Group')]
```



Density comparison of fasting by age group

```
ax = sns.kdeplot(
    data=df,
    x="Fasting",
    hue="Age Group",
```



```

    fill=True,
    common_norm=False,
    alpha=0.4
)

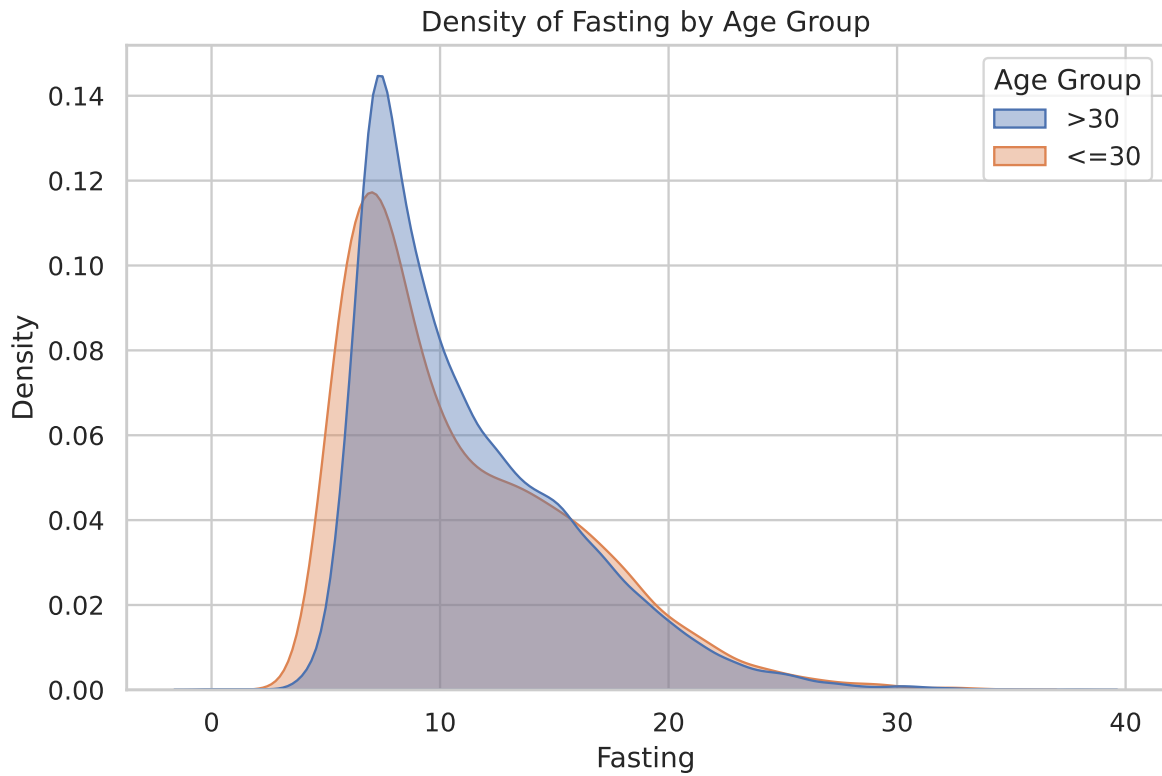
ax.set(
    xlabel="Fasting",
    ylabel="Density",
    title="Density of Fasting by Age Group"
)

```

```

[Text(0.5, 0, 'Fasting'),
 Text(0, 0.5, 'Density'),
 Text(0.5, 1.0, 'Density of Fasting by Age Group')]

```



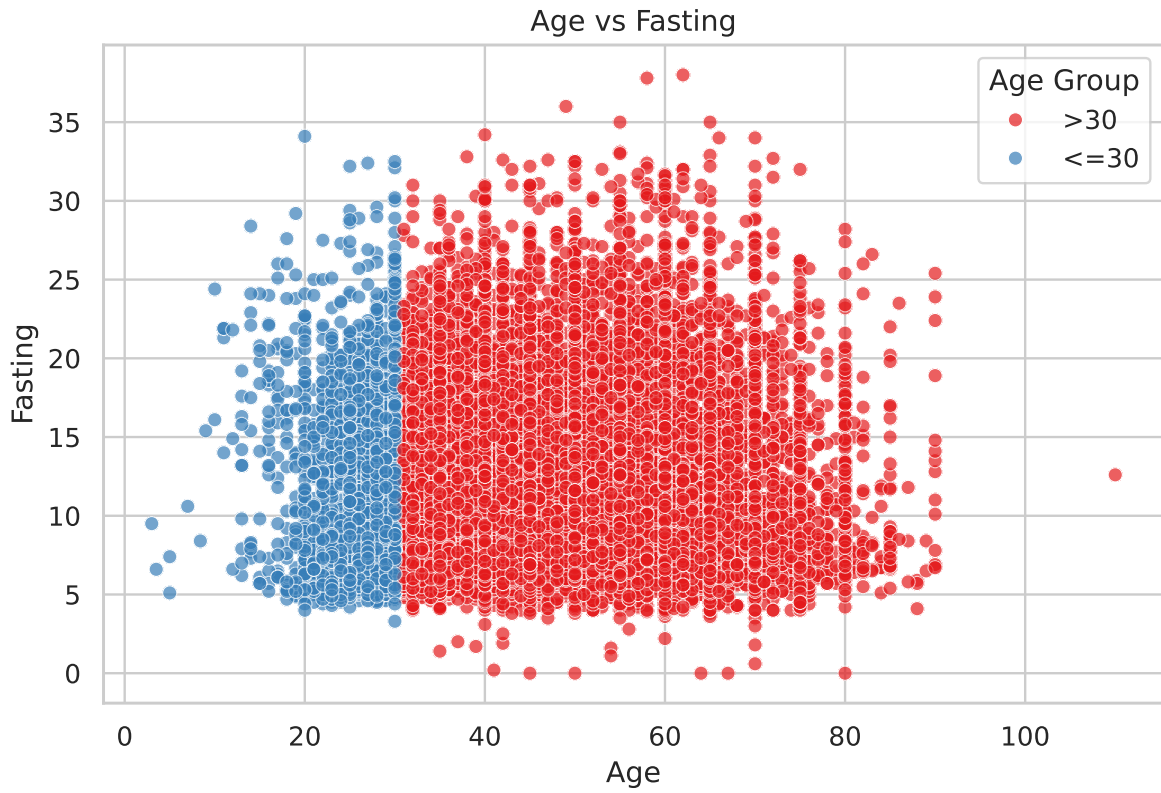
Core relationship analysis

Age vs fasting

Scatter plot

```
ax = sns.scatterplot(  
    data=df,  
    x="Age",  
    y="Fasting",  
    hue="Age Group",  
    palette="Set1",  
    alpha=0.7  
)  
  
ax.set(  
    xlabel="Age",  
    ylabel="Fasting",  
    title="Age vs Fasting"  
)
```

```
[Text(0.5, 0, 'Age'),  
Text(0, 0.5, 'Fasting'),  
Text(0.5, 1.0, 'Age vs Fasting')]
```



Mean fasting by age

```
age_fasting = (  
    df.groupby(["Age", "Age Group"], as_index=False)["Fasting"]  
        .mean()  
)  
  
ax = sns.lineplot(  
    data=age_fasting,  
    x="Age",  
    y="Fasting",  
    hue="Age Group",  
    marker="o"  
)  
  
ax.set(  
    xlabel="Age",
```

```

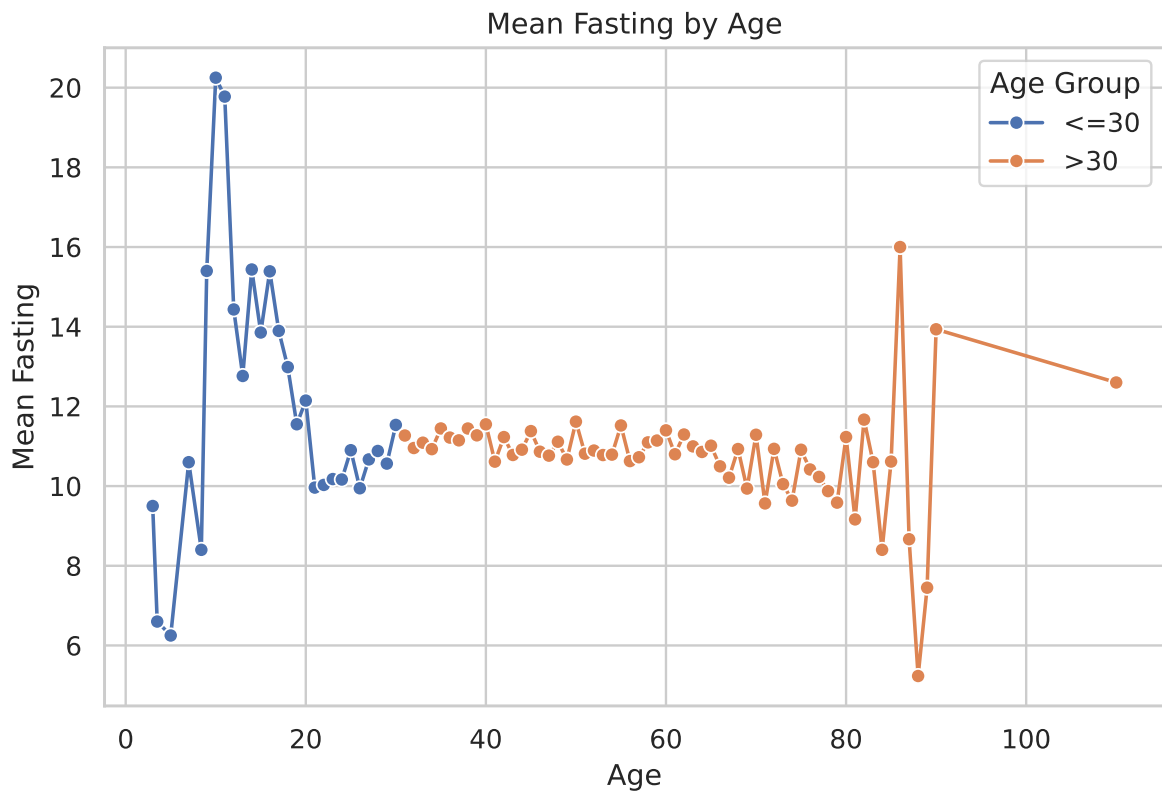
ylabel="Mean Fasting",
title="Mean Fasting by Age"
)

```

```

[Text(0.5, 0, 'Age'),
Text(0, 0.5, 'Mean Fasting'),
Text(0.5, 1.0, 'Mean Fasting by Age')]

```



BMI vs fasting

Scatter plot by age group

```

ax = sns.scatterplot(
    data=df,
    x="BMI",
    y="Fasting",

```

```

    hue="Age Group",
    alpha=0.7,
    palette="Set2"
)

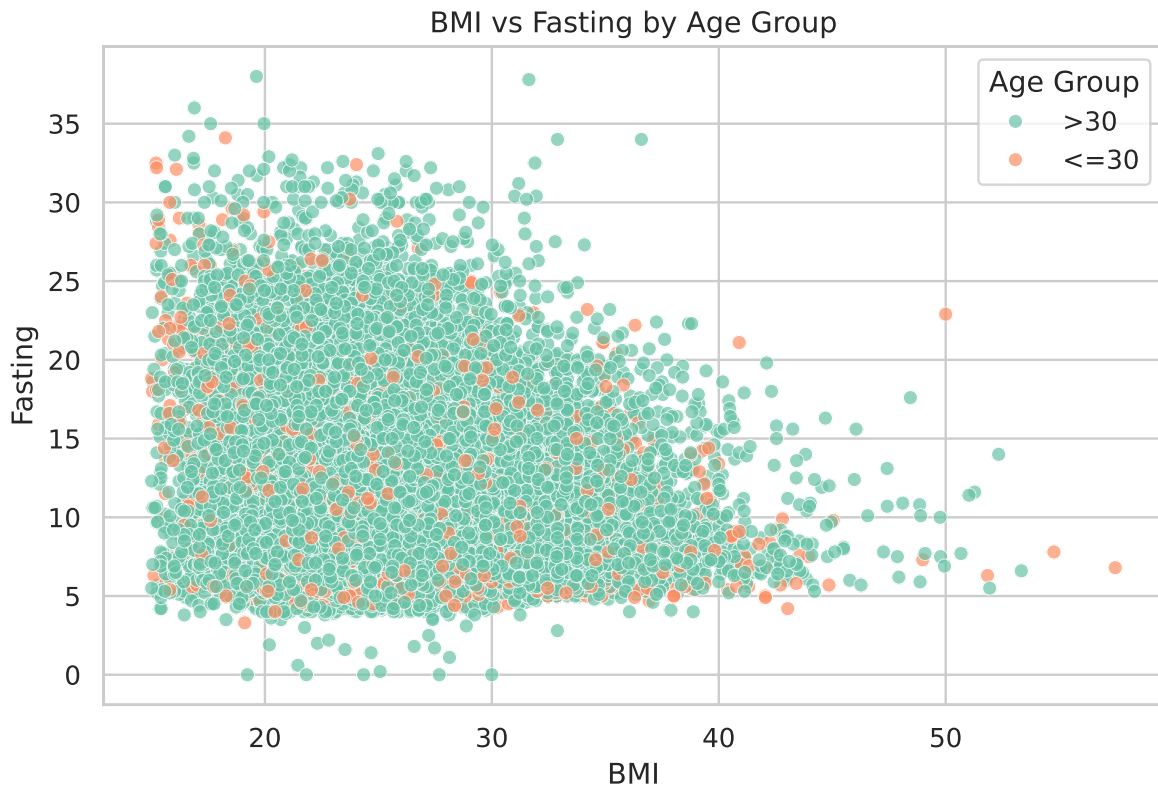
ax.set(
    xlabel="BMI",
    ylabel="Fasting",
    title="BMI vs Fasting by Age Group"
)

```

```

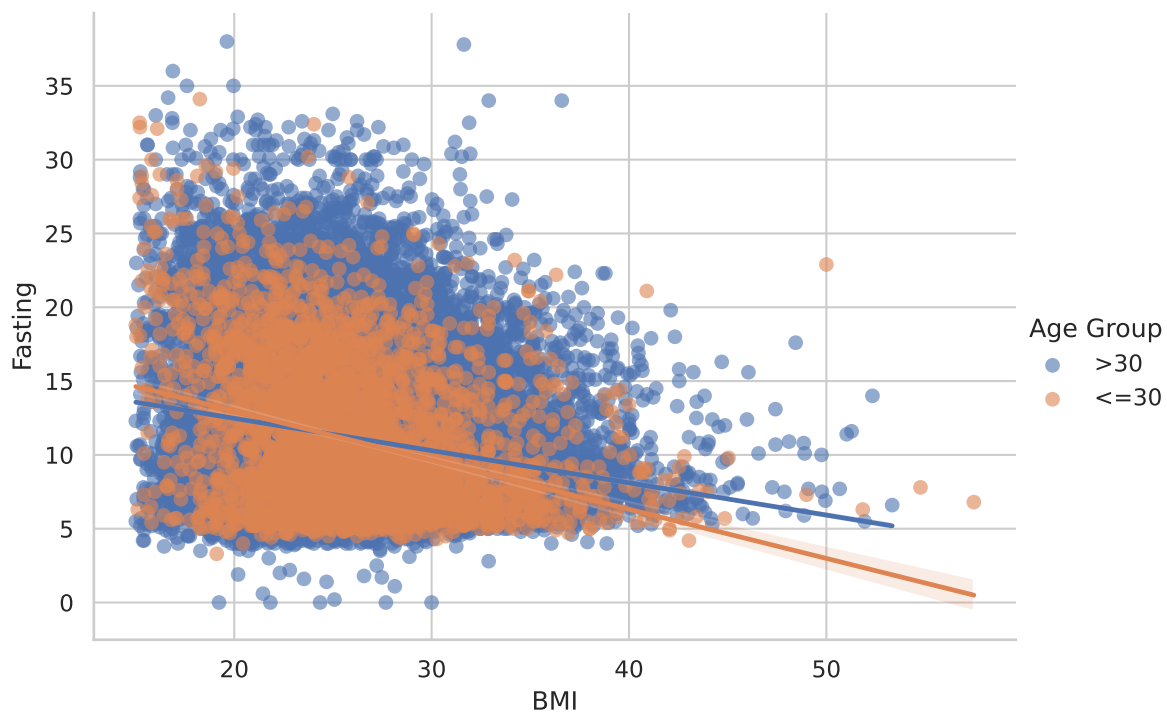
[Text(0.5, 0, 'BMI'),
 Text(0, 0.5, 'Fasting'),
 Text(0.5, 1.0, 'BMI vs Fasting by Age Group')]

```



Regression view

```
g = sns.lmplot(  
    data=df,  
    x="BMI",  
    y="Fasting",  
    hue="Age Group",  
    height=5,  
    aspect=1.4,  
    scatter_kws={"alpha": 0.6}  
)  
  
g.set_axis_labels("BMI", "Fasting")
```



Weight vs fasting

```
ax = sns.scatterplot(  
    data=df,
```

```

x="Weight (KG)",
y="Fasting",
hue="Age Group",
alpha=0.7,
palette="Set1"
)

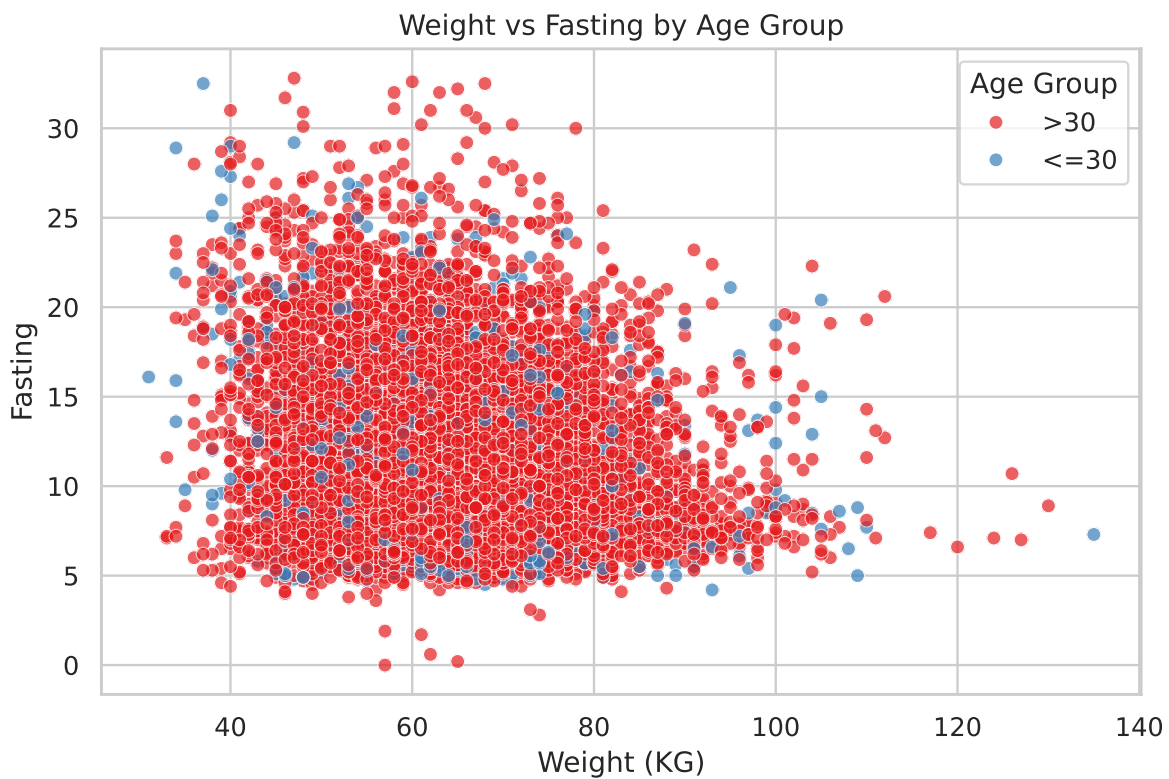
ax.set(
    xlabel="Weight (KG)",
    ylabel="Fasting",
    title="Weight vs Fasting by Age Group"
)

```

```

[Text(0.5, 0, 'Weight (KG)'),
Text(0, 0.5, 'Fasting'),
Text(0.5, 1.0, 'Weight vs Fasting by Age Group')]

```



Height vs fasting

```
ax = sns.scatterplot(  
    data=df,  
    x="Height (CM)",  
    y="Fasting",  
    hue="Age Group",  
    alpha=0.7,  
    palette="Set2"  
)  
  
ax.set(  
    xlabel="Height (CM)",  
    ylabel="Fasting",  
    title="Height vs Fasting by Age Group"  
)
```

```
[Text(0.5, 0, 'Height (CM)'),  
Text(0, 0.5, 'Fasting'),  
Text(0.5, 1.0, 'Height vs Fasting by Age Group')]
```

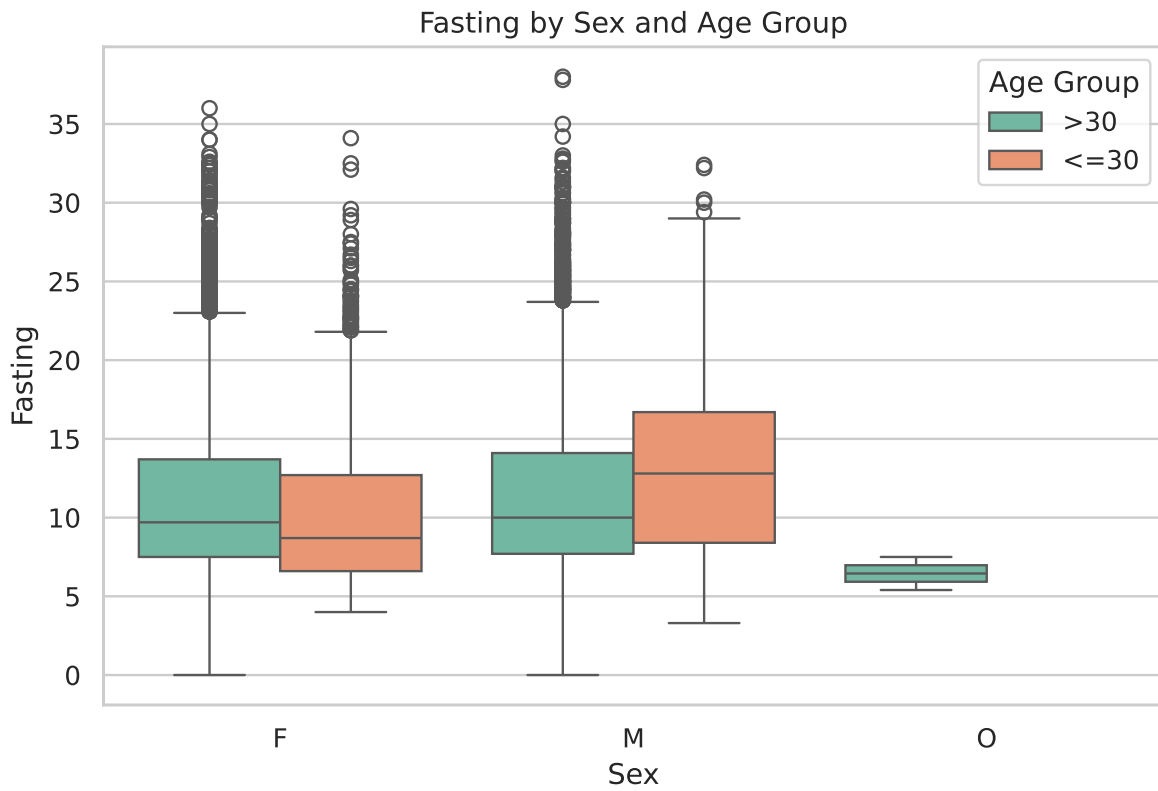



Categorical comparisons

Sex vs fasting

```
ax = sns.boxplot(  
    data=df,  
    x="Sex",  
    y="Fasting",  
    hue="Age Group",  
    palette="Set2"  
)  
  
ax.set(  
    xlabel="Sex",  
    ylabel="Fasting",  
    title="Fasting by Sex and Age Group"  
)
```

```
[Text(0.5, 0, 'Sex'),
Text(0, 0.5, 'Fasting'),
Text(0.5, 1.0, 'Fasting by Sex and Age Group')]
```

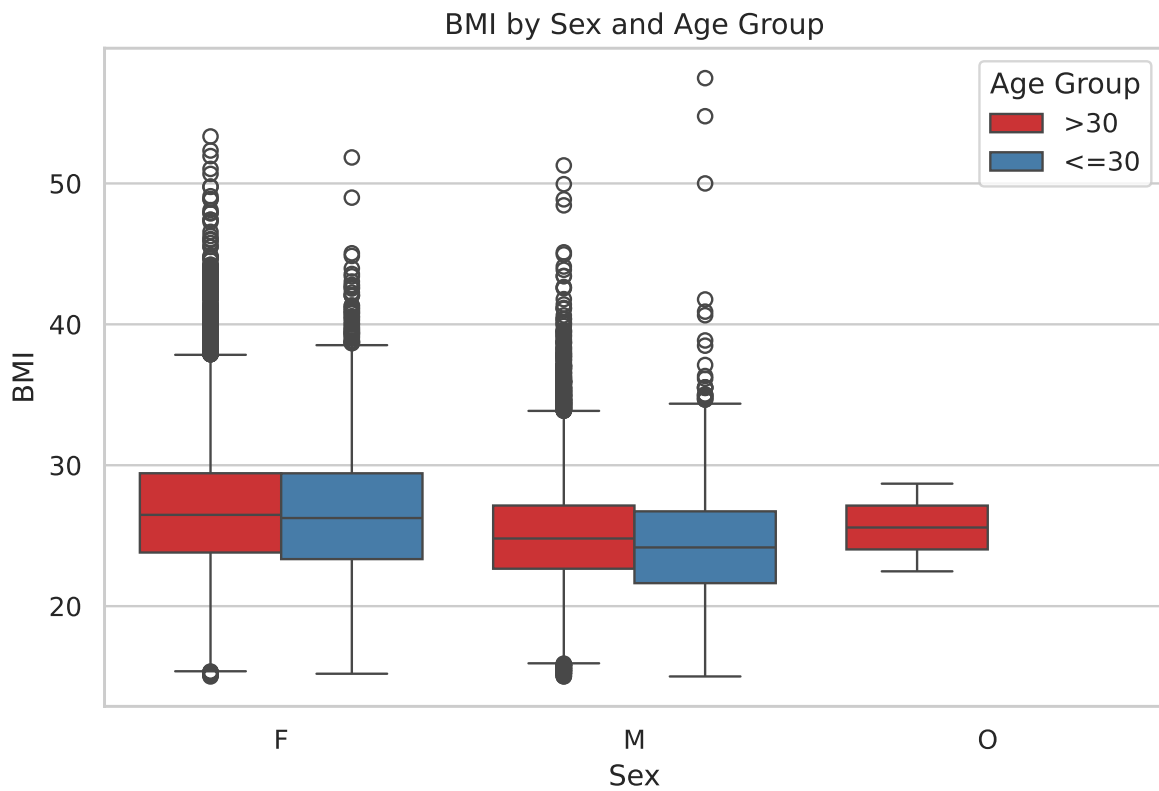


Sex vs BMI

```
ax = sns.boxplot(
    data=df,
    x="Sex",
    y="BMI",
    hue="Age Group",
    palette="Set1"
)
```

```
ax.set(
    xlabel="Sex",
    ylabel="BMI",
    title="BMI by Sex and Age Group"
)
```

```
[Text(0.5, 0, 'Sex'),
 Text(0, 0.5, 'BMI'),
 Text(0.5, 1.0, 'BMI by Sex and Age Group')]
```



Age group vs fasting

```
ax = sns.boxplot(
    data=df,
    x="Age Group",
    y="Fasting",
```

```

    hue="Age Group",
    palette="Set2",
    legend=False
)

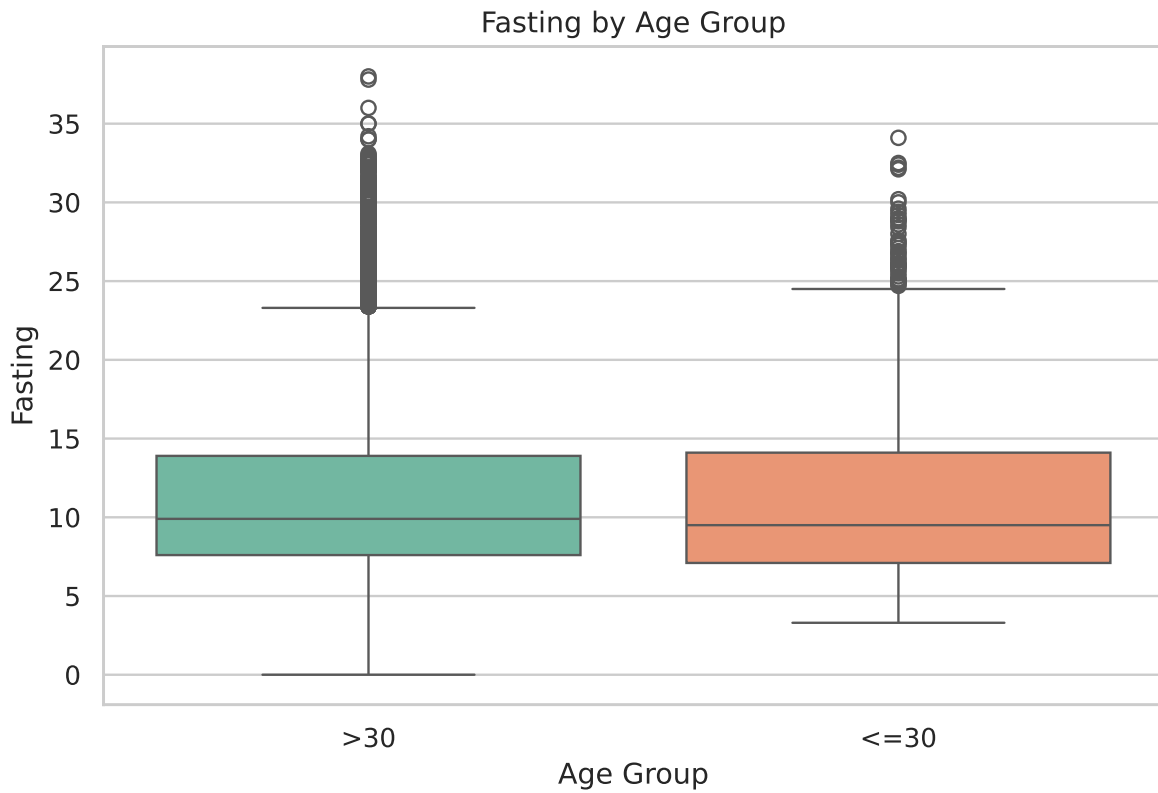
ax.set(
    xlabel="Age Group",
    ylabel="Fasting",
    title="Fasting by Age Group"
)

```

```

[Text(0.5, 0, 'Age Group'),
Text(0, 0.5, 'Fasting'),
Text(0.5, 1.0, 'Fasting by Age Group')]

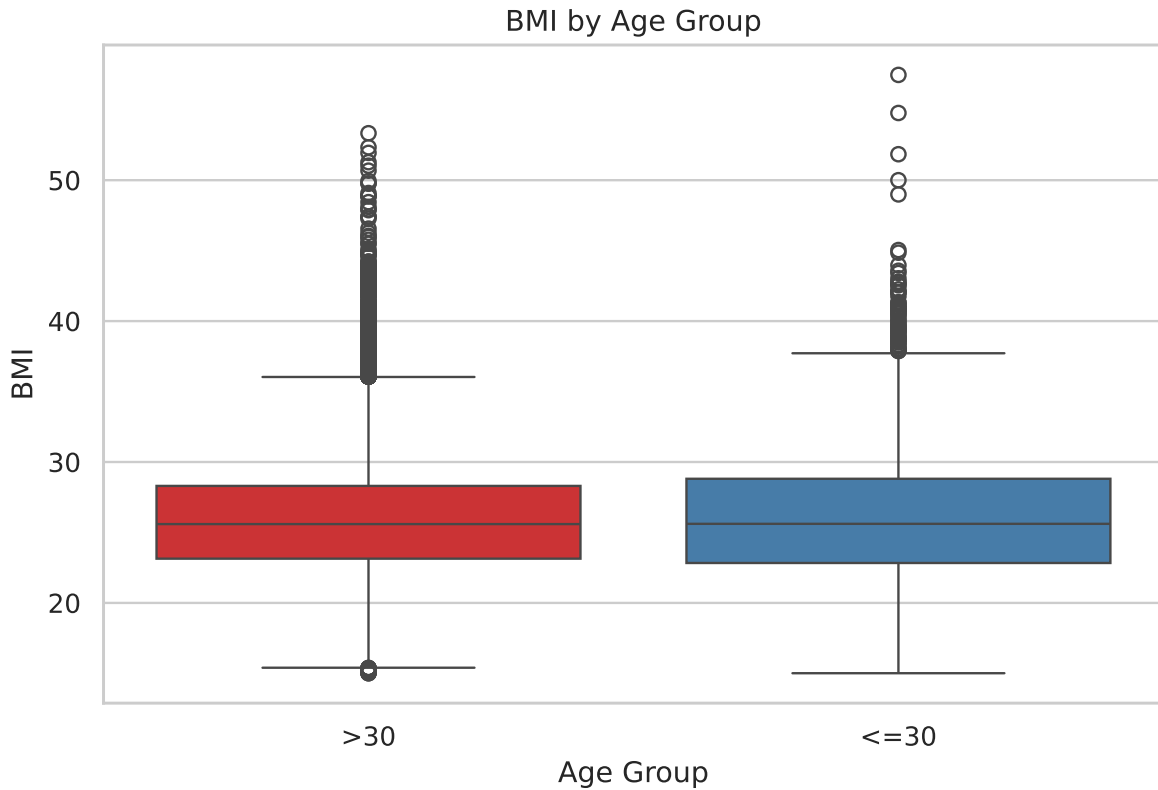
```



Age group vs BMI

```
ax = sns.boxplot(  
    data=df,  
    x="Age Group",  
    y="BMI",  
    hue="Age Group",  
    palette="Set1",  
    legend=False  
)  
  
ax.set(  
    xlabel="Age Group",  
    ylabel="BMI",  
    title="BMI by Age Group"  
)
```

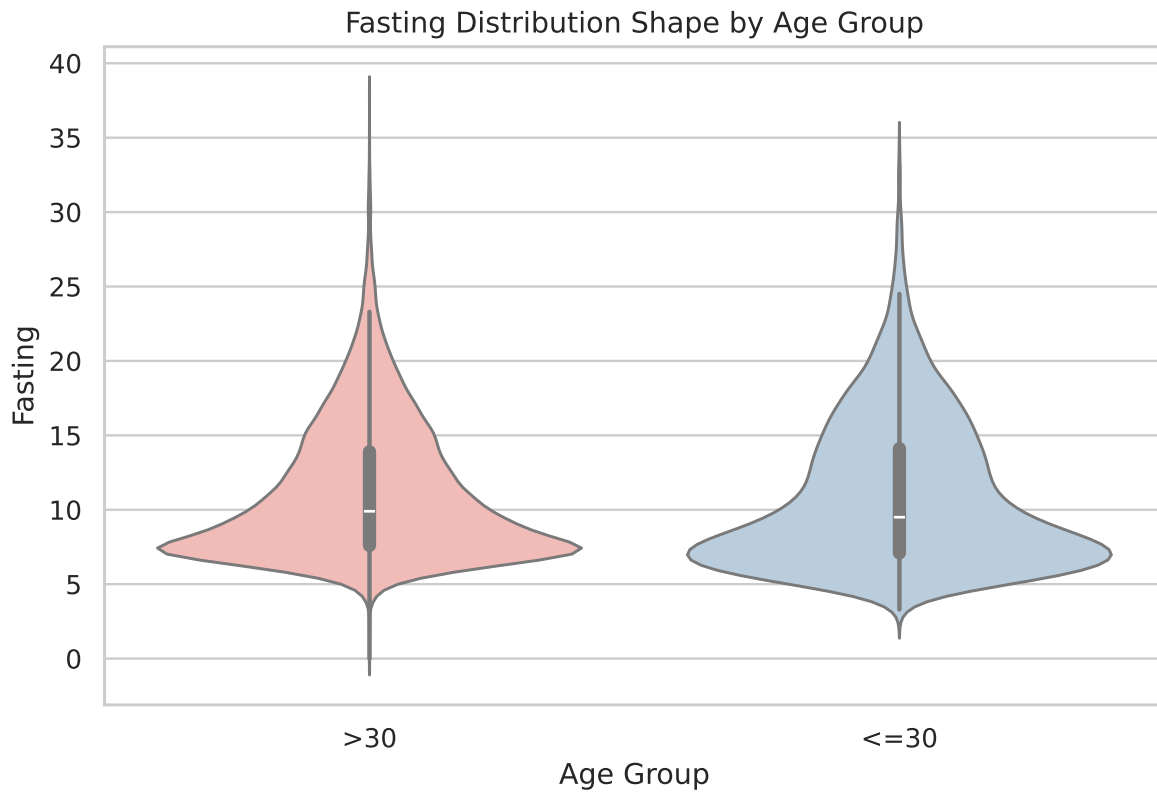
```
[Text(0.5, 0, 'Age Group'),  
Text(0, 0.5, 'BMI'),  
Text(0.5, 1.0, 'BMI by Age Group')]
```



Violin plot of fasting by age group

```
ax = sns.violinplot(  
    data=df,  
    x="Age Group",  
    y="Fasting",  
    hue="Age Group",  
    palette="Pastel1",  
    legend=False  
)  
  
ax.set(  
    xlabel="Age Group",  
    ylabel="Fasting",  
    title="Fasting Distribution Shape by Age Group"  
)
```

```
[Text(0.5, 0, 'Age Group'),
Text(0, 0.5, 'Fasting'),
Text(0.5, 1.0, 'Fasting Distribution Shape by Age Group')]
```



Diet-related analysis

This section runs only if the Diet column exists.

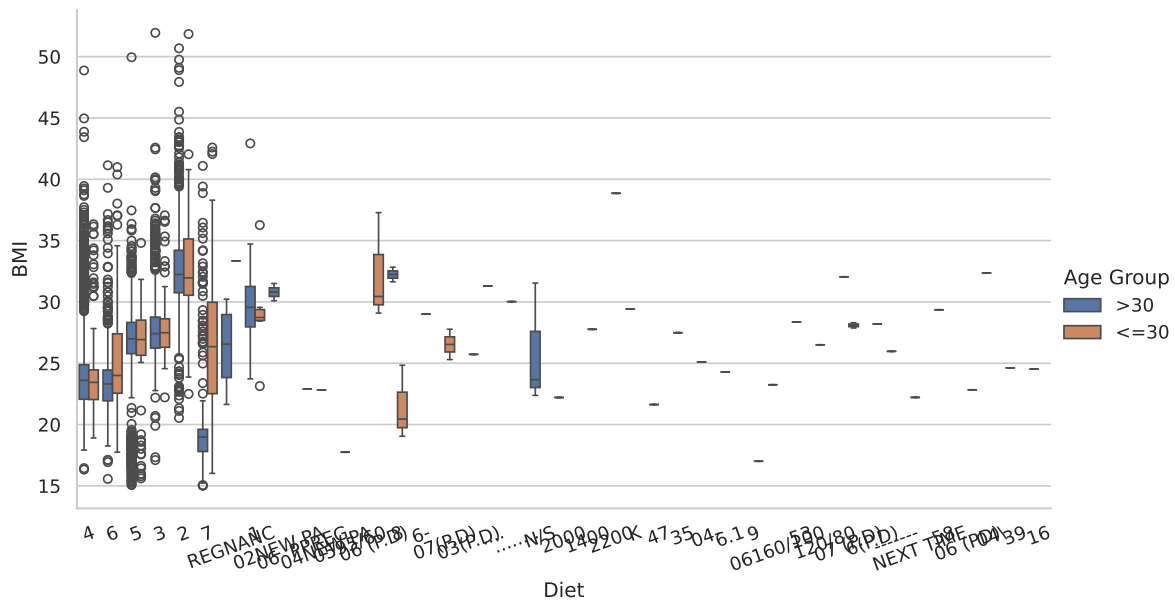
Diet frequency table

```
if "Diet" in df.columns:
    df["Diet"].value_counts(dropna=False)
else:
    print("Diet column not found in the dataset.")
```

Diet vs BMI

```
if "Diet" in df.columns:
    g = sns.catplot(
        data=df,
        x="Diet",
        y="BMI",
        hue="Age Group",
        kind="box",
        height=5,
        aspect=1.8
    )

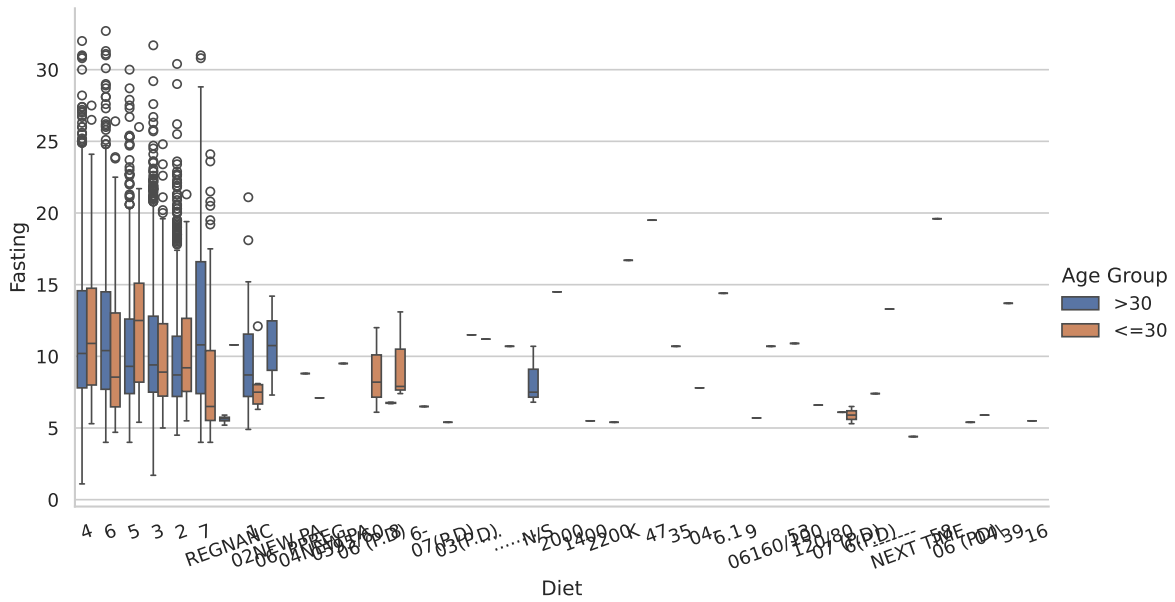
    g.set_axis_labels("Diet", "BMI")
    g.ax.tick_params(axis="x", rotation=20)
else:
    print("Diet column not found in the dataset.")
```



Diet vs fasting


```
if "Diet" in df.columns:
    g = sns.catplot(
        data=df,
        x="Diet",
        y="Fasting",
        hue="Age Group",
        kind="box",
        height=5,
        aspect=1.8
    )

    g.set_axis_labels("Diet", "Fasting")
    g.ax.tick_params(axis="x", rotation=20)
else:
    print("Diet column not found in the dataset.")
```



Diet composition across age groups

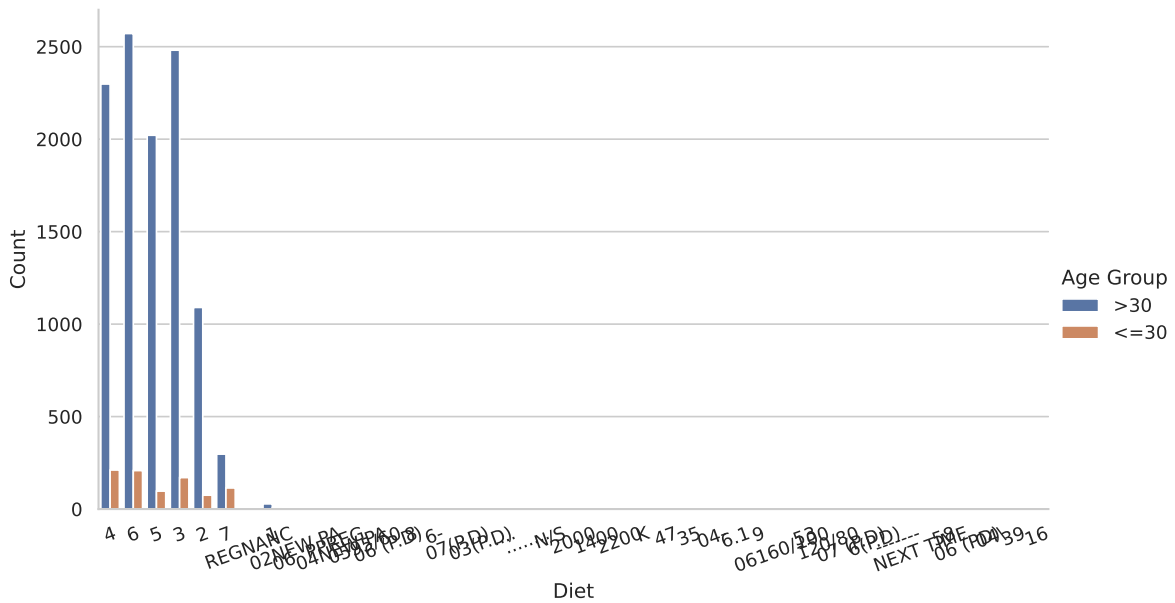
```
if "Diet" in df.columns:
    g = sns.catplot(
        data=df,
```

```

        x="Diet",
        hue="Age Group",
        kind="count",
        height=5,
        aspect=1.8
    )

    g.set_axis_labels("Diet", "Count")
    g.ax.tick_params(axis="x", rotation=20)
else:
    print("Diet column not found in the dataset.")

```



Multivariable relationship analysis

Age and sex to fasting

```
ax = sns.lineplot(
    data=df,
    x="Age",
    y="Fasting",
    hue="Sex",
```

```

    style="Age Group",
    estimator="mean"
)

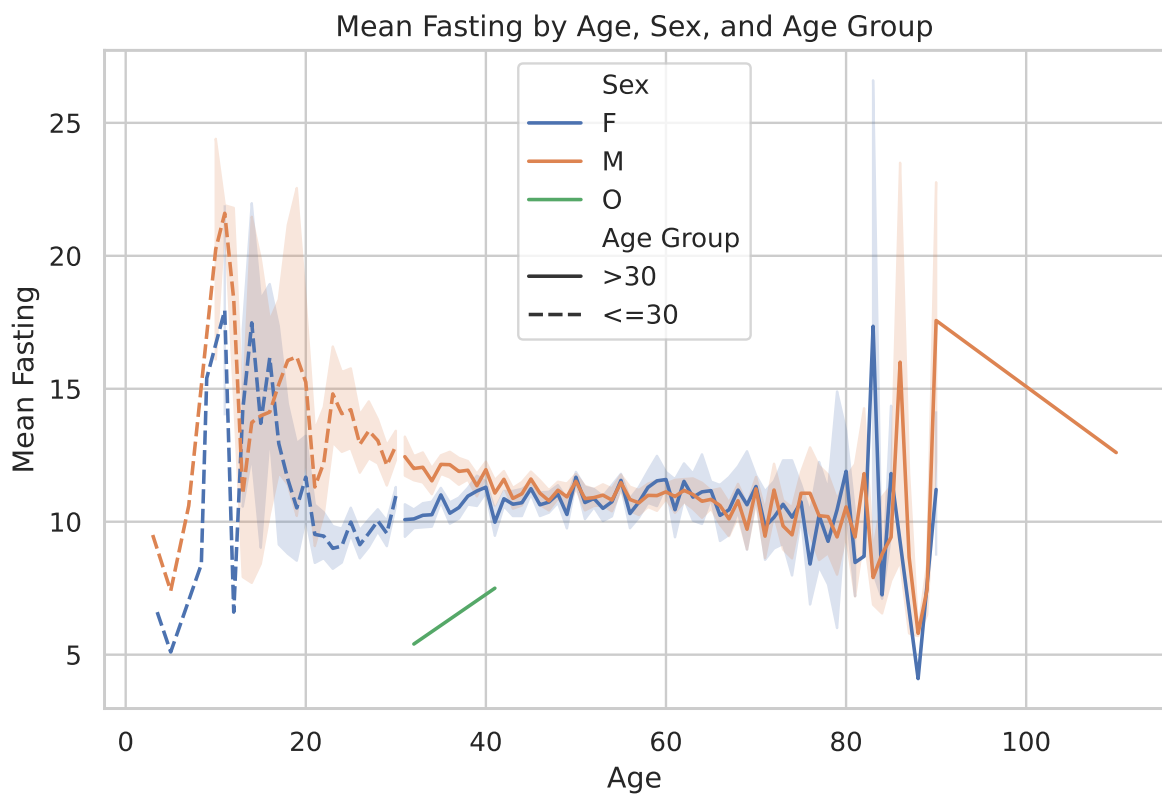
ax.set(
    xlabel="Age",
    ylabel="Mean Fasting",
    title="Mean Fasting by Age, Sex, and Age Group"
)

```

```

[Text(0.5, 0, 'Age'),
Text(0, 0.5, 'Mean Fasting'),
Text(0.5, 1.0, 'Mean Fasting by Age, Sex, and Age Group')]

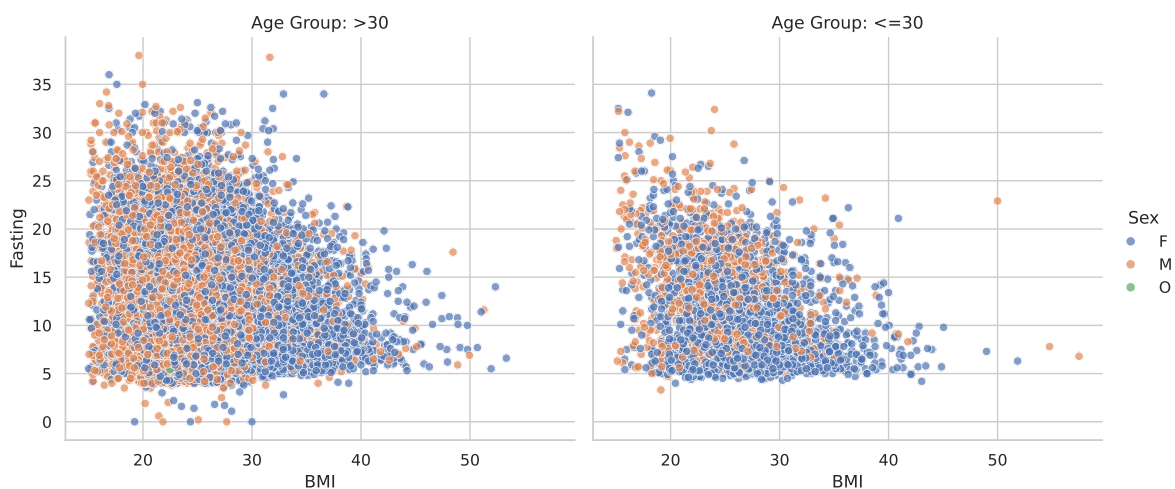
```



BMI vs fasting faceted by age group

```
g = sns.relplot(
    data=df,
    x="BMI",
    y="Fasting",
    col="Age Group",
    hue="Sex",
    kind="scatter",
    alpha=0.7,
    height=5,
    aspect=1.1
)

g.set_axis_labels("BMI", "Fasting")
g.set_titles("Age Group: {col_name}")
```



Weight vs fasting faceted by age group

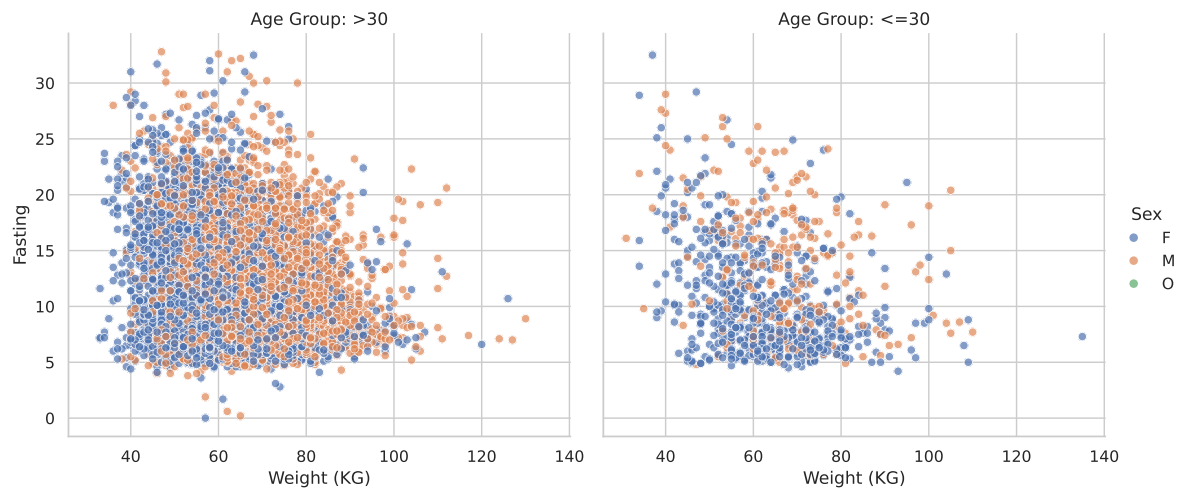
```
g = sns.relplot(
    data=df,
    x="Weight (KG)",
    y="Fasting",
    col="Age Group",
    hue="Sex",
    kind="scatter",
    alpha=0.7,
    height=5,
    aspect=1.1
)
```

```

    hue="Sex",
    kind="scatter",
    alpha=0.7,
    height=5,
    aspect=1.1
)

g.set_axis_labels("Weight (KG)", "Fasting")
g.set_titles("Age Group: {col_name}")

```



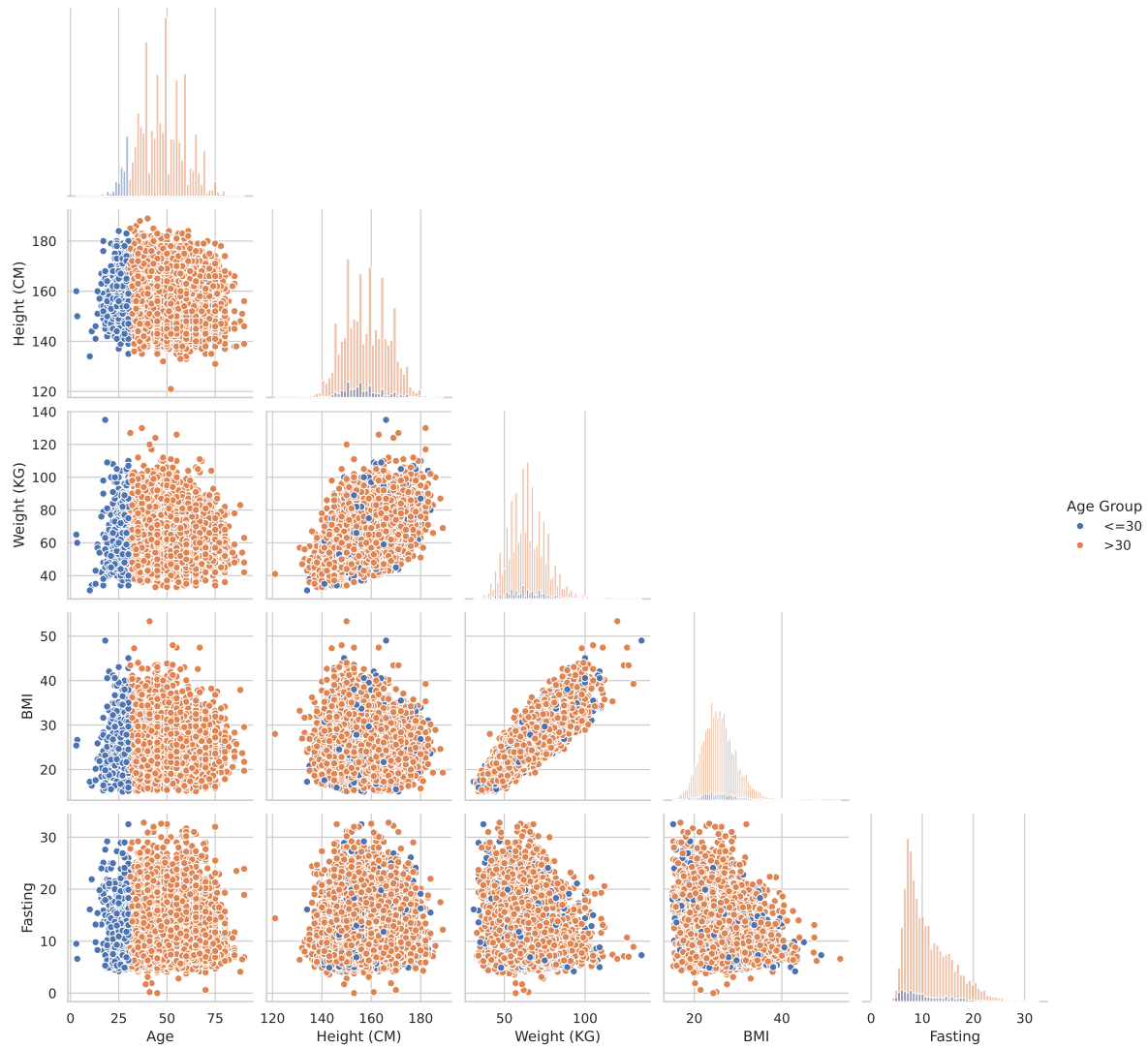
Pairwise view of core variables

```

pairplot_df = df[
    ["Age", "Height (CM)", "Weight (KG)", "BMI", "Fasting", "Age Group"]
].dropna()

sns.pairplot(
    pairplot_df,
    hue="Age Group",
    corner=True,
    diag_kind="hist"
)

```



Group comparison tables

Mean comparison across age groups

```
mean_comparison = (
    df.groupby("Age Group")[["Height (CM)", "Weight (KG)", "BMI", "Fasting"]]
    .mean()
    .round(2)
    .sort_index()
```

```
)
```

```
mean_comparison
```

	Height (CM)	Weight (KG)	BMI	Fasting
Age Group				
<=30	156.88	63.94	26.04	10.96
>30	158.13	64.43	25.94	11.18

Median comparison across age groups

```
median_comparison = (  
    df.groupby("Age Group")[["Height (CM)", "Weight (KG)", "BMI", "Fasting"]]  
    .median()  
    .round(2)  
    .sort_index()  
)
```

```
median_comparison
```

	Height (CM)	Weight (KG)	BMI	Fasting
Age Group				
<=30	155.0	63.0	25.61	9.5
>30	158.0	64.0	25.59	9.9

Sex-wise mean fasting within each age group

```
sex_age_fasting = (  
    df.groupby(["Age Group", "Sex"])["Fasting"]  
    .mean()  
    .round(2)  
    .unstack()  
)
```

```
sex_age_fasting
```

Sex	F	M	O
Age Group			
<=30	10.13	13.13	NaN
>30	11.07	11.29	6.45

BMI category by age group

```
bmi_age_table = pd.crosstab(df["BMI Category"], df["Age Group"])
bmi_age_table
```

Age Group	<=30	>30
BMI Category		
Underweight	156	827
Normal	1465	16658
Overweight	1340	16423
Obese	663	5976

Data Exploration and Fasting Glucose Quality Assessment

Data Loading and Preparation

```
import os
import pandas as pd
import geopandas as gpd
from dotenv import load_dotenv
load_dotenv()
```

True

```
# Show all rows
pd.set_option('display.max_rows', None)

# Show all columns
pd.set_option('display.max_columns', None)

# Show full cell content (stops long text from getting cut off)
pd.set_option('display.max_colwidth', None)
```

```
load_dotenv()

dataset_path = os.getenv("CLEANED_DATASET_PATH")
if dataset_path is None:
    raise ValueError("CLEANED_DATASET_PATH is not set in the environment.")

df = pd.read_excel(dataset_path)
df.head()
```

	Unnamed: 0	ID	Entry Date	Age	Sex	Address	Height (CM)	Weight
0	0	K-225859	2025-02-25 00:00:00	50.0	F	MIRESHARAI CTG	NaN	NaN
1	1	K-225401	2025-02-11 00:00:00	35.0	M	LOHAGORA CTG	NaN	NaN
2	2	K-225407	2025-02-11 00:00:00	55.0	M	SANDWIP CTG	NaN	NaN
3	3	K-225413	2025-02-11 00:00:00	50.0	M	ANOWARA CTG	NaN	NaN
4	4	K-225655	2025-02-19 00:00:00	57.0	M	MIRESHARAI CTG	NaN	NaN

```
total_population = len(df)
total_population_in_cc = df.groupby("City Corporation").count()['ID']['Chattogram City Corpor
```

```
df.groupby("Thana")["ID"].count().reset_index(name="Total_IDs")
```

	Thana	Total_IDs
0	Agrabad	1
1	Akbar Shah	1456
2	Anwara	1677
3	Bakalia	606
4	Bandar	423
5	Bandarban Sadar	3
6	Banshkhali	1817
7	Barkal	1
8	Bayazid Bostami	1062
9	Bhujpur	149
10	Boalkhali	1703
11	Chakaria	12
12	Chandanaish	999
13	Chandgaon	1306
14	Chandina	1
15	Chatkhil	1
16	Chawk Bazar	125
17	Chouddagram	1
18	Companyganj	1
19	Cox's Bazar Sadar	2
20	Cumilla Adarsha Sadar	1
21	Double Mooring	1174
22	EPZ	26
23	Fatikchhari	1899
24	Firingibazar	1
25	Fouzdarhat	1

	Thana	Total_IDs
26	Halishahar	1745
27	Hathazari	1836
28	Hatia	1
29	Jorargonj	132
30	Kaptai	4
31	Karnaphuli	1108
32	Kawkhali	1
33	Khagrachhari Sadar	7
34	Khulshi	1402
35	Kobirhat	1
36	Kotwali	1249
37	Lalkhan Bazar	5
38	Lohagara	765
39	Manikchhari	2
40	Matiranga	1
41	Mirsharai	2433
42	Moheshkhali	8
43	Muradnagar	1
44	Nabinagar	1
45	New Mooring	1
46	Noakhali Sadar	9
47	Pahartali	1820
48	Panchlaish	956
49	Patenga	253
50	Patharghata	1
51	Patiya	2050
52	Ramgarh	1
53	Rangamati Sadar	2
54	Rangunia	2081
55	Raozan	2177
56	Sadarghat	133
57	Sandwip	2081
58	Satkania	1443
59	Senbag	1
60	Shahjadpur	1
61	Shahrasti	1
62	Sitakunda	4707
63	Sonagazi	1
64	Ukhia	2

```
df["Thana_clean"] = (
    df["Thana"]
    .str.strip()
    .str.lower()
    .str.replace(r"\s*thana$", "", regex=True)
)
```

```
mapping = {
    # Fatikchhari
    "fatikchari": "Fatikchhari",
    "fatickchori": "Fatikchhari",
    "fatikchhari": "Fatikchhari",

    # Banshkhali
    "bashkhali": "Banshkhali",
    "banshkhali": "Banshkhali",

    # Bakalia
    "bakolia": "Bakalia",
    "bakalia": "Bakalia",

    # Bayazid Bostami
    "baizid": "Bayazid Bostami",
    "bayazid": "Bayazid Bostami",
    "baizid bostami": "Bayazid Bostami",
    "bayezid bostami": "Bayazid Bostami",
    "bayazid bostami": "Bayazid Bostami",

    # Double Mooring
    "doublemooring": "Double Mooring",
    "double mooring": "Double Mooring",

    # Chawk Bazar
    "chowkbazar": "Chawk Bazar",
    "chawkbazar": "Chawk Bazar",

    # Pahartali
    "pahartoli": "Pahartali",
    "pahartali": "Pahartali",

    # Sitakunda
    "sitakundu": "Sitakunda",
```

```

"sitakunda": "Sitakunda",

# Chakaria
"chokoria": "Chakaria",
"chakaria": "Chakaria",

# Mirsharai
"mirsarai": "Mirsharai",
"mirsharai": "Mirsharai",

# Karnaphuli
"karnafuly": "Karnaphuli",
"karnafuli": "Karnaphuli",
"karnaphuli": "Karnaphuli",

# Halishahar
"halisahar": "Halishahar",
"halishahar": "Halishahar",

# Jorargonj
"jorarganj": "Jorargonj",
"zorargonj": "Jorargonj",
"jorargonj": "Jorargonj",

# Akbar Shah
"akbarshah": "Akbar Shah",
"akbar shah": "Akbar Shah",

# Patenga
"potenga": "Patenga",
"patenga": "Patenga",

# EPZ
"epz": "EPZ",

# Khulshi
"khulshi": "Khulshi",

# Kotwali
"kotwali": "Kotwali",

# Panchlaish

```

```

"panchlaish": "Panchlaish",

# Chandgaon
"chandgaon": "Chandgaon",

# Bandor / Bandar
"bandar": "Bandar",

# Anwara
"anowara": "Anwara",
"anwara": "Anwara",

# Sitakunda duplicate handled already
# Raozan
"raozan": "Raozan",

# Rangunia
"rangunia": "Rangunia",

# Boalkhali
"boalkhali": "Boalkhali",

# Sandwip
"sandwip": "Sandwip",

# Lohagara
"lohagara": "Lohagara",

# Hathazari
"hathazari": "Hathazari",

# Cox's Bazar Sadar
"cox's bazar sadar": "Cox's Bazar Sadar",

# Khagrachhari Sadar
"khagrachari sadar": "Khagrachhari Sadar",
"khagrachhari sadar": "Khagrachhari Sadar",
}

```

```
df["Thana_normalized"] = df["Thana_clean"].map(mapping)
```

```
df["Thana_normalized"] = df["Thana_normalized"].fillna(
    df["Thana_clean"].str.title()
)
```

```
# overwrite original column with normalized values
```

```
df["Thana"] = df["Thana_normalized"]
```

```
# drop helper columns
```

```
df = df.drop(columns=["Thana_clean", "Thana_normalized"])
```

```
df.groupby("Thana")["ID"].count().reset_index(name="Total_IDs")
```

	Thana	Total_IDs
0	Agrabad	1
1	Akbar Shah	1456
2	Anwara	1677
3	Bakalia	606
4	Bandar	423
5	Bandarban Sadar	3
6	Banshkhali	1817
7	Barkal	1
8	Bayazid Bostami	1062
9	Bhujpur	149
10	Boalkhali	1703
11	Chakaria	12
12	Chandanaish	999
13	Chandgaon	1306
14	Chandina	1
15	Chatkhil	1
16	Chawk Bazar	125
17	Chouddagram	1
18	Companyganj	1
19	Cox's Bazar Sadar	2
20	Cumilla Adarsha Sadar	1
21	Double Mooring	1174
22	EPZ	26
23	Fatikchhari	1899
24	Firingibazar	1
25	Fouzdarhat	1
26	Halishahar	1745

	Thana	Total_IDs
27	Hathazari	1836
28	Hatia	1
29	Jorargonj	132
30	Kaptai	4
31	Karnaphuli	1108
32	Kawkhali	1
33	Khagrachhari Sadar	7
34	Khulshi	1402
35	Kobirhat	1
36	Kotwali	1249
37	Lalkhan Bazar	5
38	Lohagara	765
39	Manikchhari	2
40	Matiranga	1
41	Mirsharai	2433
42	Moheshkhali	8
43	Muradnagar	1
44	Nabinagar	1
45	New Mooring	1
46	Noakhali Sadar	9
47	Pahartali	1820
48	Panchlaish	956
49	Patenga	253
50	Patharghata	1
51	Patiya	2050
52	Ramgarh	1
53	Rangamati Sadar	2
54	Rangunia	2081
55	Raozan	2177
56	Sadarghat	133
57	Sandwip	2081
58	Satkania	1443
59	Senbag	1
60	Shahjadpur	1
61	Shahrasti	1
62	Sitakunda	4707
63	Sonagazi	1
64	Ukhia	2


```
print("Total Population:", total_population)
print("Population in CC:", total_population_in_cc)
print("Total Population outside CC:", total_population-total_population_in_cc)
```

```
Total Population: 43777
Population in CC: 13880
Total Population outside CC: 29897
```

```
df.to_excel(dataset_path)
```

Thana-Level Population Distribution and Age Group Analysis

Overview

This document presents a reproducible analysis workflow for examining population records by **Thana**, **City Corporation classification**, and **Age Group**. The experiment focuses on cleaning location labels, separating records into areas inside and outside the City Corporation, summarizing population counts, and calculating percentage distributions.

The analysis uses a cleaned Excel dataset loaded from an environment variable. The workflow is designed to support transparent reporting, verification, and future replication without hard-coding the input file path.

Objectives

The main objectives of this experiment are to:

- Load the cleaned dataset from a configured environment path.
- Remove records from Thanas that are outside the intended analysis scope.
- Standardize inconsistent or alternative Thana names through mapping.
- Classify Thanas into City Corporation and non-City Corporation groups.
- Create an age-based grouping using a threshold of 30 years.
- Produce total population counts and Thana-wise summaries.
- Calculate percentage distributions for City Corporation, outside City Corporation, and overall dataset shares.
- Run a verification check to confirm whether all records are accounted for in the final grouping.

Dataset Loading and Environment Setup

The workflow begins by importing the required Python libraries and loading environment variables. The dataset path is expected to be stored in the `CLEANED_DATASET_PATH` environment variable.

```
import os
import pandas as pd
import numpy as np
from dotenv import load_dotenv
load_dotenv()
```

True

Display Configuration

The following display settings are configured to make exploratory outputs easier to inspect. These options ensure that all rows, columns, and long cell values are visible when printed or displayed in the notebook output.

```
# Show all rows
pd.set_option('display.max_rows', None)

# Show all columns
pd.set_option('display.max_columns', None)

# Show full cell content (stops long text from getting cut off)
pd.set_option('display.max_colwidth', None)
```

Initial Data Loading and Scope Filtering

The cleaned dataset is loaded from the Excel file specified in the environment variable. A predefined list of Thanas is then excluded from the analysis. These excluded values represent locations that are not part of the intended analytical scope for this experiment.

Records with missing **Thana** values are also removed because the location field is essential for all subsequent grouping, classification, and percentage calculations.

```
path = os.getenv("CLEANED_DATASET_PATH")
if path is None:
    raise ValueError("CLEANED_DATASET_PATH not set")

df = pd.read_excel(path)

values = ["Senbag", "Shahjadpur", "Barkal", "Chandina", "Chatkhil",
```

```

        "Chouddagram", "Companyganj", "Cox's Bazar Sadar",
        "Cumilla Adarsha Sadar", "Jorargonj", "Kawkhali",
        "Khagrachhari Sadar", "Kobirhat", "Manikchhari",
        "Matiranga", "Moheshkhali", "Muradnagar", "Nabinagar",
        "Noakhali Sadar", "Ramgarh", "Rangamati Sadar",
        "Shahrasti", "Sonagazi", "Ukhia", "Hatia", "Chakaria"]

df = df.loc[~df['Thana'].isin(values)]

df.dropna(subset=["Thana"], inplace=True)

```

City Corporation and Outside City Corporation Classification

Two reference lists are created to classify Thanas into two analytical groups:

- **Inside City Corporation**, representing Thanas within the City Corporation area.
- **Outside City Corporation**, representing Thanas outside the City Corporation but still within the analysis boundary.

These lists are later used to split the dataset and calculate separate summaries for each group.

```

thanas_outside_cc = ["Mirsharai", "Fatikchhari", "Raozan", "Sitakunda", "Hathazari", "Boalkha",
thanas_in_cc = ["Bayejid Bostami", "Chandgaon", "Bakalia", "Kotwali", "Double Mooring", "Pat

```

Standardizing Thana Names

Some records contain alternative spellings, informal names, administrative variants, or localized labels for the same Thana. A mapping dictionary is used to standardize these names into the classification labels used in the analysis.

This step is important because inconsistent naming can split the same geographic area into multiple categories and distort count-based summaries.

```

mapping = {
    "Firingibazar": "Kotwali",
    "EPZ": "Chittagong Port",
    "Lalkhan Bazar": "Khulshi",
    "Patharghata": "Kotwali",
    "Kaptai": "Rangunia",
    "Fouzdarhat": "Sitakunda",

```

```

    "New Mooring": "Chittagong Port",
    "Agrabad": "Double Mooring",
    "Karnaphuli": "Patiya",
    "Bhujpur": "Fatikchhari",
    "Chawk Bazar": "Panchlaish",
    "Bandar": "Chittagong Port",
    "Anwara": "Anowara",
    "Sadarghat": "Patiya",
    "Akbar Shah": "Pahartali",
    "Bayazid Bostami": "Bayejid Bostami",
    "Bandarban Sadar": "Satkania"
}

```

The mapping is applied to the **Thana** column. Any value not found in the mapping dictionary is retained as its original value.

```
df["Thana"] = df["Thana"].map(mapping).fillna(df["Thana"])
```

Reference List Size Check

The number of Thanas in each reference list is printed to confirm the expected coverage of the City Corporation and outside City Corporation groups.

```

print(thanas_in_cc.__len__())
print(thanas_outside_cc.__len__())

```

```
11
```

```
14
```

Unique Thana Inspection

This step prints all unique Thana names available in the filtered and standardized dataset. It helps identify whether any unexpected, unmapped, misspelled, or out-of-scope Thana names remain after cleaning.

```

thanas = set(df["Thana"].tolist())

for thana in thanas:
    print(thana)

print(f"\nTotal {thanas.__len__()} thanas")

```

Satkania
 Banshkhali
 Khulshi
 Patiya
 Fatikchhari
 Double Mooring
 Sandwip
 Bayejid Bostami
 Pahartali
 Anowara
 Sitakunda
 Boalkhali
 Panchlaish
 Hathazari
 Raozan
 Halishahar
 Chandanaish
 Lohagara
 Chandgaon
 Patenga
 Mirsharai
 Bakalia
 Kotwali
 Rangunia
 Chittagong Port

Total 25 thanas

Age Group Creation

A new categorical variable, **Age Group**, is created using the following rule:

- ≤ 30 for records where age is less than or equal to 30.
- > 30 for records where age is greater than 30.

This age grouping supports comparison of younger and older population segments across Thanas and City Corporation classifications.

```
df["Age Group"] = np.where(df["Age"] <= 30, "<=30", ">30")

df["Age Group"].value_counts()

df.groupby(["Age Group", "Thana"]).size()
```

Age Group	Thana	
<=30	Anowara	141
	Bakalia	58
	Banshkhali	173
	Bayejid Bostami	102
	Boalkhali	129
	Chandanaish	79
	Chandgaon	111
	Chittagong Port	38
	Double Mooring	91
	Fatikchhari	152
	Halishahar	139
	Hathazari	145
	Khulshi	151
	Kotwali	84
	Lohagara	55
	Mirsharai	173
	Pahartali	331
	Panchlaish	75
	Patenga	25
	Patiya	281
	Rangunia	162
	Raozan	147
	Sandwip	133
	Satkania	131
	Sitakunda	478
>30	Anowara	1536
	Bakalia	548
	Banshkhali	1644
	Bayejid Bostami	960
	Boalkhali	1574
	Chandanaish	920
	Chandgaon	1195

Chittagong Port	412
Double Mooring	1084
Fatikchhari	1896
Halishahar	1606
Hathazari	1691
Khulshi	1256
Kotwali	1167
Lohagara	710
Mirsharai	2260
Pahartali	2945
Panchlaish	1006
Patenga	228
Patiya	3010
Rangunia	1923
Raozan	2030
Sandwip	1948
Satkania	1315
Sitakunda	4230

dtype: int64

Total Population Count

The total number of records remaining after filtering and standardization is calculated. This value acts as the denominator for global percentage calculations later in the workflow.

```
total_records = len(df)
print("Total Population",total_records)
```

Total Population 42678

Creating City Corporation and Outside City Corporation Subsets

The dataset is divided into two filtered DataFrames:

- `df_cc`: records belonging to Thanas inside the City Corporation.
- `df_outside`: records belonging to Thanas outside the City Corporation.

Creating these subsets simplifies downstream calculations and keeps the analysis more readable.


```
# Pre-filter DataFrames for cleaner execution
df_cc = df[df["Thana"].isin(thanas_in_cc)]
df_outside = df[df["Thana"].isin(thanas_outside_cc)]
```

Record Counts by Classification and Thana

This section calculates total records inside the City Corporation, total records outside the City Corporation, and individual Thana-level counts for both groups.

```
# Total records in City Corporation
total_cc_records = len(df_cc)

# Total records for each individual thana in City Corporation
thana_counts_in_cc = df_cc["Thana"].value_counts()

# Total records Outside City Corporation
total_outside_cc_records = len(df_outside)

# Total records for each individual thana outside City Corporation
thana_counts_outside_cc = df_outside["Thana"].value_counts()
```

Age Group Counts by Classification and Thana

The following grouped summaries are calculated:

- Total City Corporation records by age group.
- City Corporation Thana-level records by age group.
- Outside City Corporation Thana-level records by age group.
- Total outside City Corporation records by age group.

These grouped outputs support both aggregate and location-specific demographic interpretation.

```
# Total records in City Corporation grouped by Age Group
cc_records_by_age = df_cc.groupby("Age Group").size()

# Total records for each thana in CC grouped by Age Group
cc_thana_records_by_age = df_cc.groupby(["Thana", "Age Group"]).size()
```

```
# Total records for each thana outside CC grouped by Age Group
outside_thana_records_by_age = df_outside.groupby(["Thana", "Age Group"]).size()

# Total records outside CC grouped by Age Group
outside_cc_records_by_age = df_outside.groupby("Age Group").size()
```

City Corporation Summary

This section prints the total number of records inside the City Corporation and the distribution of those records by age group.

```
# Total records in CC
total_cc = len(df[df["Thana"].isin(thanas_in_cc)])
print(f"Total records in CC: {total_cc}")

# Total records in CC by Age Group
cc_age_counts = df[df["Thana"].isin(thanas_in_cc)]["Age Group"].value_counts()
print("\nTotal records in CC for each age group:")
print(cc_age_counts.to_string())
```

Total records in CC: 13612

Total records in CC for each age group:

```
Age Group
>30      12407
<=30     1205
```

Outside City Corporation Summary

This section reports record counts for each Thana outside the City Corporation. It also creates a Thana-by-age-group table, where each row represents a Thana and each column represents an age group.

```
# -----
# 1. Total records outside CC for each Thana
# -----
print("      TOTAL RECORDS OUTSIDE CC FOR EACH THANA      ")
print(thana_counts_outside_cc.to_string())
```

```

print("\n")

# -----
# 2. Total records outside CC for each Thana by Age Group
# -----

print("    RECORDS OUTSIDE CC BY THANA AND AGE GROUP    ")
# .unstack() moves the 'Age Group' index level into tidy columns
thana_age_table = outside_thana_records_by_age.unstack(fill_value=0)
print(thana_age_table.to_string())

```

```

TOTAL RECORDS OUTSIDE CC FOR EACH THANA
Thana
Sitakunda      4708
Patiya         3291
Mirsharai      2433
Raozan         2177
Rangunia       2085
Sandwip        2081
Fatikchhari    2048
Hathazari      1836
Banshkhali     1817
Boalkhali      1703
Anowara        1677
Satkania       1446
Chandanaish    999
Lohagara       765

```

```

RECORDS OUTSIDE CC BY THANA AND AGE GROUP
Age Group  <=30  >30
Thana
Anowara    141   1536
Banshkhali 173   1644
Boalkhali  129   1574
Chandanaish 79    920
Fatikchhari 152  1896
Hathazari  145  1691
Lohagara   55   710
Mirsharai  173  2260
Patiya     281  3010
Rangunia   162  1923

```

Raozan	147	2030
Sandwip	133	1948
Satkania	131	1315
Sitakunda	478	4230

City Corporation Percentage Breakdown

This section calculates the share of City Corporation records relative to the full dataset. It also calculates the age group distribution within the City Corporation subset.

```
# Calculate the total records across the entire dataset
total_dataset_records = len(df)

print("                PERCENTAGE BREAKDOWNS                ")

# 1. Total percentage in CC based on total number of records
if total_dataset_records > 0:
    cc_percentage = (total_cc_records / total_dataset_records) * 100
    print(f"Total records in CC: {cc_percentage:.2f}% of the entire dataset")
else:
    print("Total records in CC: 0.00% (Dataset is empty)")

print("-" * 50)

# 2. Percentage of each age group based on total records in CC
print("Percentage of each age group within CC:")
if total_cc_records > 0:
    cc_age_percentage = (cc_records_by_age / total_cc_records) * 100
    for age_group, percentage in cc_age_percentage.items():
        print(f"    * Age Group {age_group}: {percentage:.2f}%")
else:
    print("    * No records available in CC to calculate age percentages.")
```

```
                PERCENTAGE BREAKDOWNS
Total records in CC: 31.89% of the entire dataset
-----
Percentage of each age group within CC:
    * Age Group <=30: 8.85%
    * Age Group >30: 91.15%
```

Age Group Percentages Within City Corporation Thanas

This section calculates the internal age distribution for each City Corporation Thana. Each row is normalized by its own total so that the percentages show the age group composition within that specific Thana.

```
print("    PERCENTAGE OF AGE GROUPS IN EACH THANA (CC)    ")

# 1. Pivot the counts into a table (Thanas as rows, Age Groups as columns)
cc_thana_age_table = cc_thana_records_by_age.unstack(fill_value=0)

# 2. Divide each row by its total sum to get proportions, then multiply by 100
cc_thana_age_pct = cc_thana_age_table.div(cc_thana_age_table.sum(axis=1), axis=0) * 100

# 3. Print the formatted result rounded to 2 decimal places
print(cc_thana_age_pct.round(2).to_string())
```

PERCENTAGE OF AGE GROUPS IN EACH THANA (CC)		
Age Group	<=30	>30
Thana		
Bakalia	9.57	90.43
Bayejid Bostami	9.60	90.40
Chandgaon	8.50	91.50
Chittagong Port	8.44	91.56
Double Mooring	7.74	92.26
Halishahar	7.97	92.03
Khulshi	10.73	89.27
Kotwali	6.71	93.29
Pahartali	10.10	89.90
Panchlaish	6.94	93.06
Patenga	9.88	90.12

Outside City Corporation Percentage Breakdown

This section reports two types of Thana-level percentage calculations for areas outside the City Corporation:

- Percentage based on the total number of records outside the City Corporation.
- Percentage based on the total population of the entire dataset.

It then calculates age group percentages within each outside-City-Corporation Thana.

```
print("      THANA RECORD PERCENTAGES (OUTSIDE CC)      ")

if total_outside_cc_records > 0:
    # Option A: Based on total records OUTSIDE CC
    thana_pct_outside_total = (thana_counts_outside_cc / total_outside_cc_records) * 100
    print("A. Percentage based on total records OUTSIDE CC:")
    print(thana_pct_outside_total.round(2).to_string())

    print("-" * 50)

    # Option B: Based on the ENTIRE dataset's total records
    thana_pct_global_total = (thana_counts_outside_cc / total_dataset_records) * 100
    print("B. Percentage based on the ENTIRE dataset population:")
    print(thana_pct_global_total.round(2).to_string())
else:
    print("No records found outside CC.")

print("      PERCENTAGE OF AGE GROUPS IN EACH THANA (OUTSIDE)")

# Pivot, divide across rows (axis=0) to get internal distributions, and multiply by 100
outside_thana_age_table = outside_thana_records_by_age.unstack(fill_value=0)

if not outside_thana_age_table.empty:
    outside_thana_age_pct = outside_thana_age_table.div(outside_thana_age_table.sum(axis=1),
    print(outside_thana_age_pct.round(2).to_string())
else:
    print("No age group records found outside CC.")
```

```
      THANA RECORD PERCENTAGES (OUTSIDE CC)
A. Percentage based on total records OUTSIDE CC:
Thana
Sitakunda      16.20
Patiya         11.32
Mirsharai      8.37
Raozan         7.49
Rangunia       7.17
Sandwip        7.16
Fatikchhari    7.05
```

Hathazari	6.32
Banshkhali	6.25
Boalkhali	5.86
Anowara	5.77
Satkania	4.97
Chandanaish	3.44
Lohagara	2.63

B. Percentage based on the ENTIRE dataset population:

Thana

Sitakunda	11.03
Patiya	7.71
Mirsharai	5.70
Raozan	5.10
Rangunia	4.89
Sandwip	4.88
Fatikchhari	4.80
Hathazari	4.30
Banshkhali	4.26
Boalkhali	3.99
Anowara	3.93
Satkania	3.39
Chandanaish	2.34
Lohagara	1.79

PERCENTAGE OF AGE GROUPS IN EACH THANA (OUTSIDE)

Age Group	<=30	>30
Thana		
Anowara	8.41	91.59
Banshkhali	9.52	90.48
Boalkhali	7.57	92.43
Chandanaish	7.91	92.09
Fatikchhari	7.42	92.58
Hathazari	7.90	92.10
Lohagara	7.19	92.81
Mirsharai	7.11	92.89
Patiya	8.54	91.46
Rangunia	7.77	92.23
Raozan	6.75	93.25
Sandwip	6.39	93.61
Satkania	9.06	90.94
Sitakunda	10.15	89.85

Inside City Corporation Thana Percentage Breakdown

This section reports two types of percentage calculations for Thanas inside the City Corporation:

- Percentage based on total records inside the City Corporation.
- Percentage based on total records in the full dataset.

These outputs help distinguish between local distribution within the City Corporation and global contribution to the overall dataset.

```
print("          THANA RECORD PERCENTAGES (INSIDE CC)          ")

if total_cc_records > 0:
    # Option A: Based on total records INSIDE CC
    thana_pct_cc_total = (thana_counts_in_cc / total_cc_records) * 100
    print("A. Percentage based on total records INSIDE CC:")
    print(thana_pct_cc_total.round(2).to_string())

    print("-" * 50)

    # Option B: Based on the ENTIRE dataset's total records
    thana_pct_global_total = (thana_counts_in_cc / total_dataset_records) * 100
    print("B. Percentage based on the ENTIRE dataset population:")
    print(thana_pct_global_total.round(2).to_string())
else:
    print("No records found inside CC.")
```

```
          THANA RECORD PERCENTAGES (INSIDE CC)
A. Percentage based on total records INSIDE CC:
Thana
Pahartali          24.07
Halishahar         12.82
Khulshi            10.34
Chandgaon           9.59
Kotwali            9.19
Double Mooring      8.63
Panchlaish          7.94
Bayejid Bostami     7.80
Bakalia             4.45
Chittagong Port     3.31
```


Patenga	1.86
---------	------

B. Percentage based on the ENTIRE dataset population:

Thana

Pahartali	7.68
Halishahar	4.09
Khulshi	3.30
Chandgaon	3.06
Kotwali	2.93
Double Mooring	2.75
Panchlaish	2.53
Bayejid Bostami	2.49
Bakalia	1.42
Chittagong Port	1.05
Patenga	0.59

Global Population Share and Verification

The final section calculates the City Corporation share of the total dataset and the individual outside-City-Corporation Thana shares. These percentages are then summed to verify whether the classification accounts for the entire filtered dataset.

If the combined percentage does not equal 100%, the output warns that some records may still contain missing, unmapped, or unclassified Thana values.

```
# Ensure we have the global total
total_population = len(df)

print("          GLOBAL POPULATION SHARE BREAKDOWN          ")

# 1. Percentage of records in CC based on total population
cc_pct_global = (len(df[df["Thana"].isin(thanas_in_cc)]) / total_population) * 100
print(f"Total City Corporation (CC) Share: {cc_pct_global:.2f}%\n")

print("-----")
print("Individual Thana Shares (Outside CC):")
# 2. Percentage of records in each thana outside CC based on total population
outside_thana_pct_global = (df[df["Thana"].isin(thanas_outside_cc)]["Thana"].value_counts()
print(outside_thana_pct_global.round(2).to_string())
```

```

print("                                VERIFICATION CHECK                                ")

# Summing CC total + individual outside thana percentages
grand_total_percentage = cc_pct_global + outside_thana_pct_global.sum()
print(f"Combined Sum of Percentages: {grand_total_percentage:.2f}%")

if round(grand_total_percentage, 2) == 100.00:
    print(" Check Passed: Perfect 100%! All data is accounted for.")
else:
    print(" Check Warning: Does not equal 100%. You may have missing (NaN) values or unmapped")

```

GLOBAL POPULATION SHARE BREAKDOWN

Total City Corporation (CC) Share: 31.89%

Individual Thana Shares (Outside CC):

Thana

Sitakunda	11.03
Patiya	7.71
Mirsharai	5.70
Raozan	5.10
Rangunia	4.89
Sandwip	4.88
Fatikchhari	4.80
Hathazari	4.30
Banshkhali	4.26
Boalkhali	3.99
Anowara	3.93
Satkania	3.39
Chandanaish	2.34
Lohagara	1.79

VERIFICATION CHECK

Combined Sum of Percentages: 100.00%

Check Passed: Perfect 100%! All data is accounted for.

Interpretation Guide

Use the generated outputs as follows:

- **Total Population** shows the number of valid records after filtering and removing missing Thana values.
- **Total records in CC** shows how many records fall inside the City Corporation classification.
- **Outside CC Thana counts** show how the non-City-Corporation records are distributed geographically.
- **Age Group summaries** show whether records are concentrated in the ≤ 30 or > 30 group.
- **Within-group percentages** explain internal composition, such as the age distribution inside a specific Thana.
- **Global percentages** show how much each group contributes to the entire filtered dataset.
- **Verification check** confirms whether all standardized Thana records are covered by either the City Corporation list or the outside City Corporation list.

Notes for Reproducibility

Before rendering this document, ensure that:

- The `.env` file exists in the working directory or the environment variable is otherwise available.
- `CLEANED_DATASET_PATH` points to the cleaned Excel dataset.
- The dataset contains the required columns: **Thana** and **Age**.
- Required Python packages are installed: `pandas`, `numpy`, `python-dotenv`, and an Excel-reading engine such as `openpyxl`.

Expected Output

After running the document, the analysis should produce:

- A cleaned and standardized Thana list.
- Total population count after filtering.
- Counts for City Corporation and outside City Corporation areas.
- Thana-wise record counts.
- Age group counts and percentages.
- City Corporation and outside City Corporation percentage breakdowns.
- A final verification result indicating whether the grouped records sum to the full filtered dataset.